WeightCode: Installing New Capabilities into Neural Networks via Weight Programming

Bo Li[1,2,*], Taoyong Cui[3,*,†], Bob Zhang[1,‡], Mengran Li[4],
Chengyang Zhang[5], Yang Zhang[2], Pheng Ann Heng[3],
Wanli Ouyang[3], Zhenfei Yin[6,‡], Yingcheng Wu[7,‡], Ling Yang[8,‡]

[1]University of Macau [2]National University of Singapore
[3]The Chinese University of Hong Kong [4]Sun Yat-sen University
[5]Sichuan University [6]University of Oxford
[7]Stanford University [8]Peking University

[*]Equal contribution. [†]Project lead. [‡]Corresponding authors.

Code: https://github.com/Gen-Verse/WeightCode **Project Page:** Website

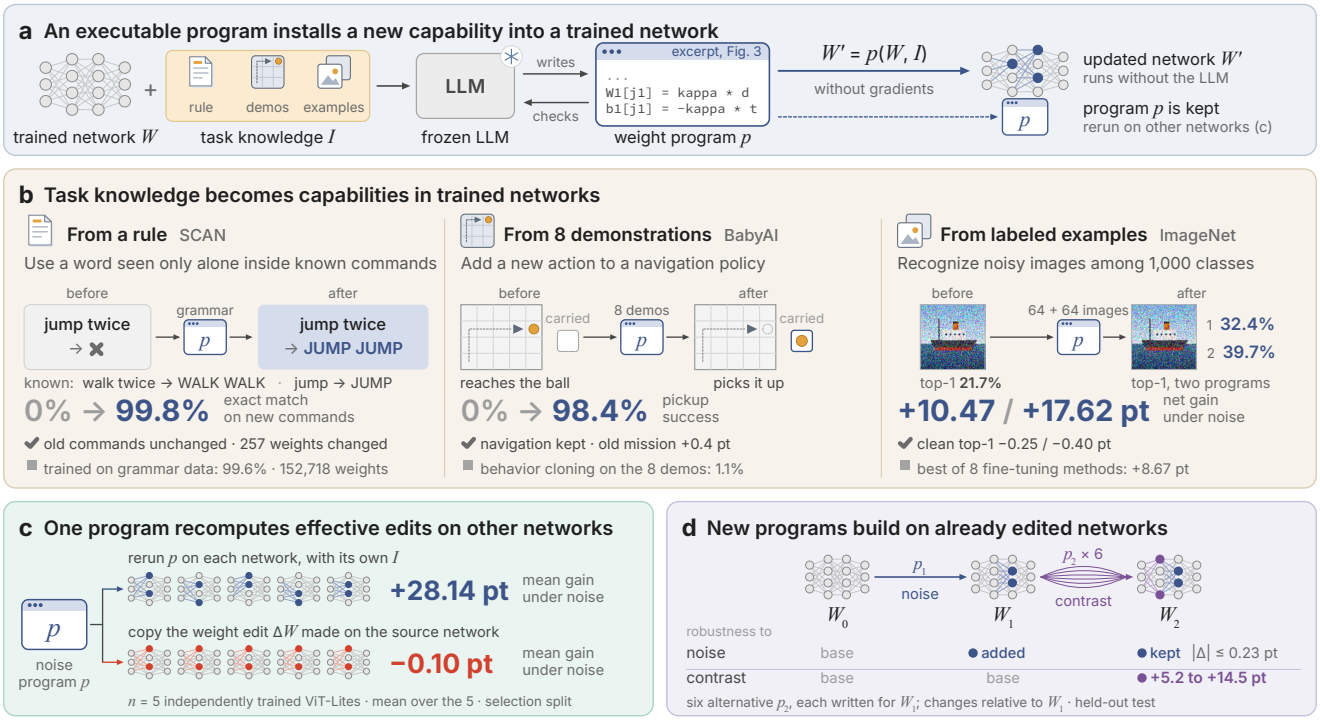


Figure 1. **WeightCode installs, reuses, and extends capabilities through weight programming.** (a) A frozen LLM uses a trained network's weights W and task knowledge I to write an executable weight program p, which computes $W' = p(W, I)$ without gradients. (b) Weight programs turn rules, demonstrations, and labeled examples into new capabilities across language, navigation, and visual recognition while preserving existing capabilities. (c) Rerunning the same program recomputes effective edits across independently trained networks, whereas copying its fixed weight edit does not reproduce the gain. (d) Programs written for an already edited network install a second capability while preserving the first.

Abstract

Neural networks encode learned capabilities in their weights, yet extending those capabilities with explicit task knowledge remains a fundamental challenge. Here we show that executable weight programs can construct and install new capabilities directly within trained neural networks. WeightCode uses a frozen language model to write executable weight programs that install new computations into trained neural networks using task knowledge and their existing weights, without gradient-based training. Once installed, the new capabilities run independently of the language model. Given a grammar or eight demonstrations, weight programs enable previously unseen command compositions (0% to 99.8% exact match) and a new pickup action (0% to 98.4% success), while preserving the original command and navigation capabilities. On ImageNet, four weight programs achieve net gains of 8.84–17.62 points while preserving clean accuracy, outperforming the best of eight fine-tuning methods under matched examples, checks, and retention constraints. In a Vision Transformer, causal interventions show that installed noise-correction gates drive robustness gains, while their selective activation preserves clean accuracy. Rerun on five independently trained Vision Transformers without further language model calls, ten weight programs retain 83–110% of their source-network gains on average, whereas copying their fixed weight edits yields negligible gains. Six programs written for an already edited network add 5.23–14.50 points of contrast robustness while retaining the previously installed noise gain. Together, these results establish weight programming as a new route to acquiring and accumulating capabilities in trained neural networks through reusable executable programs.

1. Introduction

Learning a capability during training and installing one into an already trained neural network are distinct problems. The latter requires translating task knowledge into new computations while preserving previously acquired capabilities. Unlike software, whose functionality can be extended through executable instructions, neural networks encode learned computations in distributed weights. Can executable programs directly construct new capabilities within these weights, and can subsequent programs build upon those already installed?

An executable weight program retains how a network-specific update is constructed, rather than only the numerical weight changes it produces. Fine-tuning [1, 2], model editing [3, 4], and test-time adaptation [5, 6] provide established ways to update trained networks. Meanwhile, large language models (LLMs) can generate and refine executable programs through evaluation feedback [7, 8] and have recently been shown to inspect and modify neural weights directly [9]. Four fundamental questions define the scope of weight programming:

- *Installation.* Can a weight program install a new capability into a trained network while preserving previously acquired capabilities?
- *Mechanism.* What computation does a weight program install, and how does it produce the new capability?
- *Reuse.* Can the same weight program be rerun without the LLM to construct effective edits in independently trained networks?
- *Succession.* Can a new weight program extend an already edited network while preserving capabilities installed by earlier programs?

Here we introduce WeightCode, which uses a frozen LLM to write executable weight programs that install new computations into trained neural networks (Fig. 1a). A weight program p reads the network's weights W and task knowledge I, such as a rule, demonstrations, or labeled examples, and constructs updated weights $W' = p(W, I)$. Evaluation feedback selects programs that improve the new capability while preserving existing behavior. The updated network executes the installed computation within its original architecture, without the LLM at inference. The retained program can be rerun on compatible networks to construct network-specific edits. New programs can then extend networks already modified by earlier programs.

Across language, navigation, and visual recognition, weight programs transform rules, demonstrations, and labeled examples into new capabilities (Fig. 1b, Fig. 2). On SCAN, a program modifies 257 weights to raise exact match on previously unseen command compositions from 0.0% to 99.8%; on BabyAI, eight demonstrations enable a new pickup action with 98.4% success. On ImageNet, four weight programs achieve 8.84–17.62 points of net gain under corruption while preserving clean accuracy, outperforming eight fine-tuning methods under matched examples, checks, and retention constraints. Causal interventions reveal how installed noise-correction gates produce robustness gains while their selective activation preserves clean accuracy (Fig. 3). Rerun on five independently trained Vision Transformers, ten weight programs retain 83–110% of their source-network gains on average, whereas copying fixed weight edits yields negligible gains (Fig. 1c, Fig. 4). Six second-round programs written for an already edited network add contrast robustness while retaining the previously installed noise robustness (Fig. 1d, Fig. 5). Together, these findings establish a new paradigm for extending trained neural networks: capabilities can be programmed into existing weights, reconstructed across independently trained networks, and progressively expanded through successive weight programs.

2. Related work

Existing approaches to modifying trained networks differ in how they construct, represent and reuse changes to model parameters.

Learning and retaining capabilities. Fine-tuning adds a capability by optimizing parameters on labeled data, in full or through parameter-efficient modules [1, 10], and regularized transfer keeps the new parameters close to the old ones [11]. Sequential training overwrites earlier capabilities [12, 13], and continual learning limits this forgetting by regularizing parameters [14] or outputs [2], or by replaying stored examples [15]. Test-time adaptation adjusts a network to a new input condition from unlabeled inputs, by re-estimating normalization statistics [5], minimizing prediction entropy [6, 16] or training on a self-supervised task [17].

Targeted and lifelong model editing. Model editing changes a trained network's behavior on target inputs while preserving it elsewhere: by gradient steps on a network trained to be editable [18], by learned editor networks [19], or by closed-form rewrites of layers viewed as associative memories [3, 4, 20, 21]. Lifelong editing applies edits in sequence, for example by storing each one in a discrete key-value codebook attached to a layer [22].

Composing and transferring weight changes. Task arithmetic adds and subtracts the weight differences of fine-tuned models [23], and model merging resolves the interference between such differences when it combines them [24]. Both act on models that share a pretrained starting point, and merging independently trained networks first aligns their units by permutation [25]. These methods operate on learned parameter differences, whose transfer and combination depend on the relationship between model parameterizations.

Programming neural computation and weight space. Language models have been used to discover executable algorithms, reward functions, policies, skills and training objectives [7, 8, 26–29], and symbolic search and meta-learning discover optimizers [30, 31]. Separately, work on hypernetworks, weight-space learning and program-to-network compilation has established several ways to construct or analyze neural computation through weights [32–38].

Weight-space programming. In concurrent work, Takehi et al. [9] show that a frontier LLM can read the weights of small networks, write networks from zero weights and edit trained networks, and Li et al. [39] have an LLM write an editor that refits the output weights or coefficients of trained predictors from measured feedback. These studies show that language models can inspect and modify neural weights. WeightCode shows that the programs that construct such updates, retained after the update, can be rerun on independently trained networks and extended by subsequent programs, so that the construction procedure itself becomes a reusable object.

3. WeightCode

WeightCode separates the construction of an update from its execution: a frozen LLM writes an executable program, and the program computes network-specific weight changes (Fig. 1a).

3.1. Installing a capability with a program

Weight programs. A weight program p is code that takes a trained network's weights W and task knowledge I, what is known about a new capability, and returns new weights:

$$W' = p(W, I), \qquad \Delta W = p(W, I) - W. \tag{1}$$

ΔW is the *weight edit* of one run. It depends on the network the program reads as well as on the program, so the same program run on another network computes another edit. A program may read every weight, run the network's forward pass on inputs and read I. The updated network keeps its architecture and forward code, with only its weights changed, and runs on its own at inference. The experiments impose additional implementation constraints, including sandboxed execution and a maximum edit size of 20% of the network's weights (Sec. A.1).

What a program writes. In WeightCode, the form of the update is chosen for each capability and each network. The LLM proposes candidate computations and weight modifications from the network's weights, its activations and the task knowledge, and evaluation feedback determines which candidate is kept. The programs reported here connect the new capability to computation the network already performs (Tab. 9). The Gaussian-noise program for the ViT-Lite, for example, measures how noise shifts the patch tokens, finds MLP units that are nearly idle on clean images, and rewrites two of them in each of three blocks as gates that open when a token moves along the noise direction and then subtract the measured shift (Fig. 3a). Other programs reuse the network's own representations, such as the embeddings of known primitives on SCAN or of a known instruction in the BabyAI policy.

Writing and checking. The LLM writes the program in one session, in a Python kernel that holds W, the forward pass and I. Program construction alternates between

inspecting the network, writing candidates and evaluating them. A check runs a candidate on the network and returns two counts on a check split that I does not contain: S_{new}, the inputs of the new condition that the candidate fixes minus those it breaks, and S_{old}, the same count on old inputs. A session has 8 checks.

Keeping existing behavior. Preservation is evaluated on task-specific reference conditions: clean-image accuracy for the image classifiers, the previously supported commands or mission for the sequence and navigation tasks, and the previously installed conditions for successive updates. Keeping these is part of how a program is chosen. The kept program is the checked candidate with the highest $S_{\text{new}} + S_{\text{old}}$ among those with $S_{\text{old}} \geq -20$, a loss of at most 0.5% of 4,000 clean check images (the *retention rule*). On SCAN the bound is 5 of 1,000 old commands and on BabyAI 3 of 500 old episodes. If no candidate qualifies, the network is left unchanged. The bound is applied on the check split, and the reference conditions are measured on held-out inputs (Sec. 3.3).

Feedback without gradients. Weight programs construct updates through direct computations on weights, activations and forward passes, namely weight and activation statistics, SVD and PCA, label-free closed-form fits and single-unit ablations, rather than by optimizing an objective on the network's outputs. Labeled feedback evaluates and selects candidate programs rather than fitting their values: a single-round session scores at most 8 candidate edits against the labeled examples and receives 8 checks, as each fine-tuning method tries 8 configurations and receives the same checks. Label-scored work that changes no weight, and selection inside a program's own run, are counted separately from this cap (Sec. A.1). An automatic audit and a reading of the kept programs check these rules; three exploratory sessions that use finite differences are reported separately (Sec. A.11), and Sec. A.1 lists what the protocol admits.

3.2. Rerunning and extending programs

Programs as reusable construction procedures. An executable weight program retains the procedure that constructed an update: how the network is measured, how units or directions are selected and what is written into them. Run on another compatible network, the same program remeasures that network and constructs a new edit in its weight coordinates. This matters for independently trained networks, which may implement related functions in different coordinates [25], so that a numerical edit computed on one network need not apply to another.

Copying, rerunning and successive updates. Three operations act on a program p_1 kept for a network W_0 and task knowledge I_1, which produces $W_1 = p_1(W_0, I_1)$. *Copying* adds the edit $W_1 - W_0$ unchanged to another network. *Rerunning* executes the code of p_1 unchanged and without another LLM session, while the program inspects the target network and uses that network's own task examples, so each rerun computes an edit from that network's weights and data. A *successive update* asks the LLM for a new program p_2 for the edited network W_1, which produces $W_2 = p_2(W_1, I_2)$. This new program is constructed with the edited network, the code of p_1, labeled examples of the new condition and checks that also report every earlier condition, under the retention rule for each, so that the update can account for the computations installed earlier. We rerun programs on independently trained networks of the architecture they were written for, and write second programs for edited networks.

3.3. Experimental protocol

Networks. Programs are written by Claude Opus 5.5 at maximum reasoning effort. The image networks are a Vision Transformer (ViT) [40], ViT-Lite-7/4 [41] (3.7M weights, 89.35% clean accuracy), and a ResNet-20 [42] (0.27M weights, 91.28%), both trained on 40,000 CIFAR-10 training images, and torchvision's ImageNet [43] ResNet-18 in two forms: unchanged on its 1,000 classes (11.7M weights, 69.70% on our ImageNet test images) and fine-tuned to CIFAR-10 (11.2M weights, 95.78%). The other networks are a BabyAI policy trained by imitation to go to a named object and a Transformer per SCAN split [44, 45]. Five further networks per CIFAR-10 architecture, trained from other seeds, serve reuse.

Conditions. Corruptions are the CIFAR-10-C set at severity 5 [46] and, on ImageNet, the ImageNet-C functions at the highest severity that costs at least 15 points and keeps at least 17% of the network's accuracy above chance: 3 for Gaussian noise and 4 for contrast reduction. A condition is run on a network when it costs the network at least 15 points of accuracy, which on the ViT admits Gaussian noise and contrast reduction. Table 1 summarizes what each task gives the LLM and the references, and Sec. A.1 gives training, data and sandbox details.

Task knowledge. I comes at several levels. *Labeled examples*: 64 labeled images of the new condition (16 that the network misclassifies and 48 drawn at random, with the network's predictions) and 64 labeled clean images. For the BabyAI policy, 8 demonstrations of the new mission (each step's observation, instruction and expert action) and 64 old steps. *Condition labels withheld*: the LLM receives the same condition images and the network's predictions but not their labels, and no statement of how they were chosen. The image pool and the selection are unchanged, including

the use of labels to pick the 16 misclassified images, and the clean images and the checks keep their labels. *A rule*: SCAN's published interpretation grammar with 64 old commands and, on the add-primitive split, the training row of the new primitive. For the ViT, the formula that generates the corruption and 64 labeled clean images, with no image of the condition. *A capability's name*: one sentence asking for robustness to additive Gaussian pixel noise, with no image, label or prediction, given with the 8 checks or with none, in which case the one program the LLM declares is scored once after the session. This is the only level at which the LLM is told the condition.

References and criterion. Every fine-tuning comparison fits the same labels by gradient. It has eight methods: full, last layer, LayerNorm only (BatchNorm on the ResNet-18s), LoRA [1] with rank 4 and 16, full without the clean images, full with LwF [2] and full with L2-SP [11]. Comparisons match the task examples, the number of external checks and the retention rule: each method sees the same labeled images (the method without clean images only the 64 of the condition), tries 8 configurations, receives the same 8 checks and is kept by the same rule. The methods differ in how they construct updates, including the pretrained prior of the LLM that writes each program. We also report each method's best configuration on the check split without the clean-loss bound. Eleven fixed references per image network, label fits and label-free rules including Tent [6], BN adaptation [5] on the ResNets and SAR [16] on the ViT, are kept by the same rule. Before the sessions on the ViT and the ResNet-18s we fixed a performance criterion for the comparative evaluation, the *success criterion*: a gain of at least 10 points (on ImageNet, 15% of the accuracy the condition costs: 7.0 and 8.5 points), a clean change of at least -1.0 and a net gain at least one point above the strongest reference that obeys the program limits (for most conditions, gradient training of a fixed 19% of each tensor's entries) given the labeled examples. Sessions given less information on the ViT are read against the same bar.

Splits. The held-out images form three disjoint splits per condition: the condition images of I are drawn from an evidence split (2,000 images), the checks use a check split (4,000) and analyses are read on a selection split (4,000). The clean images of I are one fixed draw from the network's training images. On ImageNet the 50,000 validation images are divided by image: 12,000 form these three splits and the pool of clean images for I, and the other 38,000 are the test set. The selection split plays no part in any session, check, retention rule or fine-tuning configuration choice. The tables report the held-out test sets: the official CIFAR-10 and CIFAR-10-C test files, the 38,000 ImageNet test images, the third of SCAN's test commands kept for the final evaluation, and held-out BabyAI episodes. Each kept network was read on them once, after every selection was final.

Metrics and reporting. *Gain* is the accuracy change on the condition, *clean change* the accuracy change on clean images, and *net gain* their sum. The mechanism analyses (Sec. 4.2) and the copied edits (Sec. 4.3) are read on the selection split, and other numbers that exist only on the selection split are marked where they appear. Each program is one LLM session on one trained network. On the ViT we ran ten sessions, three per condition on the seed-100 network and one per condition on networks trained from seeds 101 and 102, and report all of them (Tab. 7). On the ImageNet ResNet-18 we ran two sessions per condition and report both (Tab. 8). Table 3 shows the first session on each network.

4. Results

4.1. Weight programs turn task knowledge into new capabilities and keep existing behavior

A rule becomes a new primitive inside existing computation. SCAN's add-primitive split [44] trains a Transformer on every command except those that compose the primitive *jump*, which it sees only alone, and the network executes none of these compositions (Fig. 2a, Tab. 2). Given SCAN's interpretation grammar, 64 old commands and the *jump* row, the program raises exact match on them from 0.0% to 99.8% by changing 257 weights, and the old commands do not change. It moves the embedding of *jump* to the centroid of those of *walk*, *look* and *run* and adds an output row for its action, so the composition the network already computes now carries the new primitive. A rule-to-data reference, trained on commands generated from the grammar, reaches 99.6% with 152,718 changed weights, about 594 times as many, and fine-tuning on the given rows keeps at most +1.21 points of net gain (selection split). A compact rule can therefore be installed by reusing the Transformer's existing compositional computation, with 257 modified weights.

The same principle extends to a different compositional setting. On the around-right split [45], where *around* was trained only with *left*, the program writes the rule directly into 25,056 weights, as rank-one edits of attention values along the network's own right-minus-left direction, and raises exact match on commands with *around right* from 17.6% to 89.6% at -0.3 points on the old commands, where fine-tuning on the given rows keeps at most +2.61 points of net gain (selection split; the rule-to-data reference is in Fig. 2a and Tab. 2).

Eight demonstrations become a new action. A BabyAI policy [47] trained by imitation to go to a named object never uses the pickup action. Given 8 demonstrations of the mission *pick up* and 64 old steps, the program raises pickup

success from 0.0% to 98.4%, and the old mission moves by +0.4 points. The installed behavior generalizes beyond the demonstrations: success remains 98.9% on episodes whose object color and type never appear in the demonstrations (selection split). Behavior cloning on the same demonstrations reaches at most 1.1% while keeping the old mission (Fig. 2a). The program thus adds pickup to the navigation the policy already performs while retaining its original mission (Sec. 4.2).

With the same examples, programs reach larger gains than gradient training on ImageNet. On torchvision's ResNet-18 unchanged on its 1,000 ImageNet classes, every method receives the same 64 labeled images of a corruption and 64 labeled clean images, the same 8 checks and the same retention rule (Fig. 2b, Tab. 3). The two programs written for Gaussian noise reach net gains of +10.47 and +17.62 points and the two written for contrast reduction +8.84 and +14.26, against +8.67 and +3.38 for the best of eight fine-tuning methods and +8.84 and +6.00 for the best fixed reference (Tab. 8). The four programs also keep clean accuracy within 0.89 points of the original network on ImageNetV2 [48], a test set collected separately following ImageNet's procedure.

The result holds across architectures on CIFAR-10: the first programs reach net gains of +32.79 and +16.51 on the ViT-Lite, against +22.78 and +7.89 for the best fine-tuning, and +35.23 and +21.28 on the ResNet-18 fine-tuned to CIFAR-10, 3.7 and 2.1 times the best fine-tuning. The resulting edits vary with the network and the computation, from 3,846 modified weights for the ViT-Lite noise gate to 2,152,928 for the ImageNet noise correction (0.10% to 18% of each network).

Independently written programs also exceed matched fine-tuning: on three ViT-Lites, eight of ten programs written in separate sessions do so on their own network, with net gains from +6.61 to +32.79 (Fig. 2c, Tab. 7).

The program's lead is largest where existing behavior must be kept and labeled data are scarce. On the ViT-Lite under Gaussian noise, full fine-tuning first matches the program's net gain within the retention rule at 4,096 labeled corrupted images and as many clean ones, compared with 64 of each for the program (selection split, Fig. 6). At clean-loss bounds of 0.5 and 1 point on the check split, the program has the highest net gain of every method on all four CIFAR-10 instances, by +2.7 to +25.5 points; Sec. A.2 and Fig. 7 chart the full trade-off between gain and retention.

Programs can be written from the condition images without their labels. On the ViT-Lite under Gaussian noise, with the labels of the 64 noisy images withheld from the LLM and the network's own predictions in their place, the program reaches a net gain of +34.43 at a clean change of -0.04, compared with +32.79 for the session given all labels (Tab. 3). The images are selected as in the labeled setting (16 that the network misclassifies), and the clean images and the 8 checks are labeled. Given only the capability's name, programs reach +9.43 and +15.10 on the selection split (Fig. 8). Direct access to the condition labels is thus not required in this setting: the network's predictions, statistics of the images and labeled check feedback suffice to construct an effective correction.

4.2. Weight programs install computations that can be tested part by part

Weight programs produce identifiable internal computations whose causal contributions to the new behavior can be tested directly (Fig. 3; Tab. 9 describes every program and marks those tested by intervention; selection split).

The noise program writes a conditional correction into existing units. The installed correction detects a shift in the network's internal representations and subtracts a measured offset selectively from the affected tokens. By examining the supplied images and the network's activations, the LLM finds that the corruption mainly shifts the patch tokens in the early residual stream, and builds the correction on that shift; untold, it also infers additive white Gaussian pixel noise of standard deviation about 25/255 (the generating value is 0.10). The program measures this shift on synthetic noisy copies of the clean images, frees two little-used MLP units in block 2 by folding their mean output into the bias, and rewrites them as a ramp gate,

$$g(z) = \mathrm{GELU}\big(k(d^\top z - t)\big) - \mathrm{GELU}\big(k(d^\top z - t) - K\big), \tag{2}$$

which subtracts the measured offset only from tokens whose projection $d^\top z$ on a whitened noise direction lies above the clean range (Fig. 3a). Two gates of the same form act on the class token in blocks 4 and 6.

The installed gates carry the gain. Removing the block-2 gate lowers the net gain from +31.75 to +10.05, and removing all three gates returns the network to its original accuracy (Fig. 3c).

Selective activation keeps clean accuracy. The block-2 gate opens on 99.5% of noisy and 0.5% of clean patch tokens and cuts the mean noisy-minus-clean offset from 5.29 to 0.50, while the program moves clean patch tokens by 0.05 on average against 5.02 on noisy ones (Fig. 3b). Holding the gates open lowers the net gain to +19.43 and clean accuracy by 5.03 points, 15.5 on the worst class. Gate activation also tracks where the correction helps: on 25 corrupted versions of the test set, the program helps on the whole noise family and on frost, where the gate opens on 27% to 100% of patch tokens, and changes no other set by more than 0.27 points on the test sets (Fig. 9).

The gated computation matters more to the gain than the identity of the units that carry it (Fig. 3c). The gated structure keeps the gain when its parts are replaced: a gate built

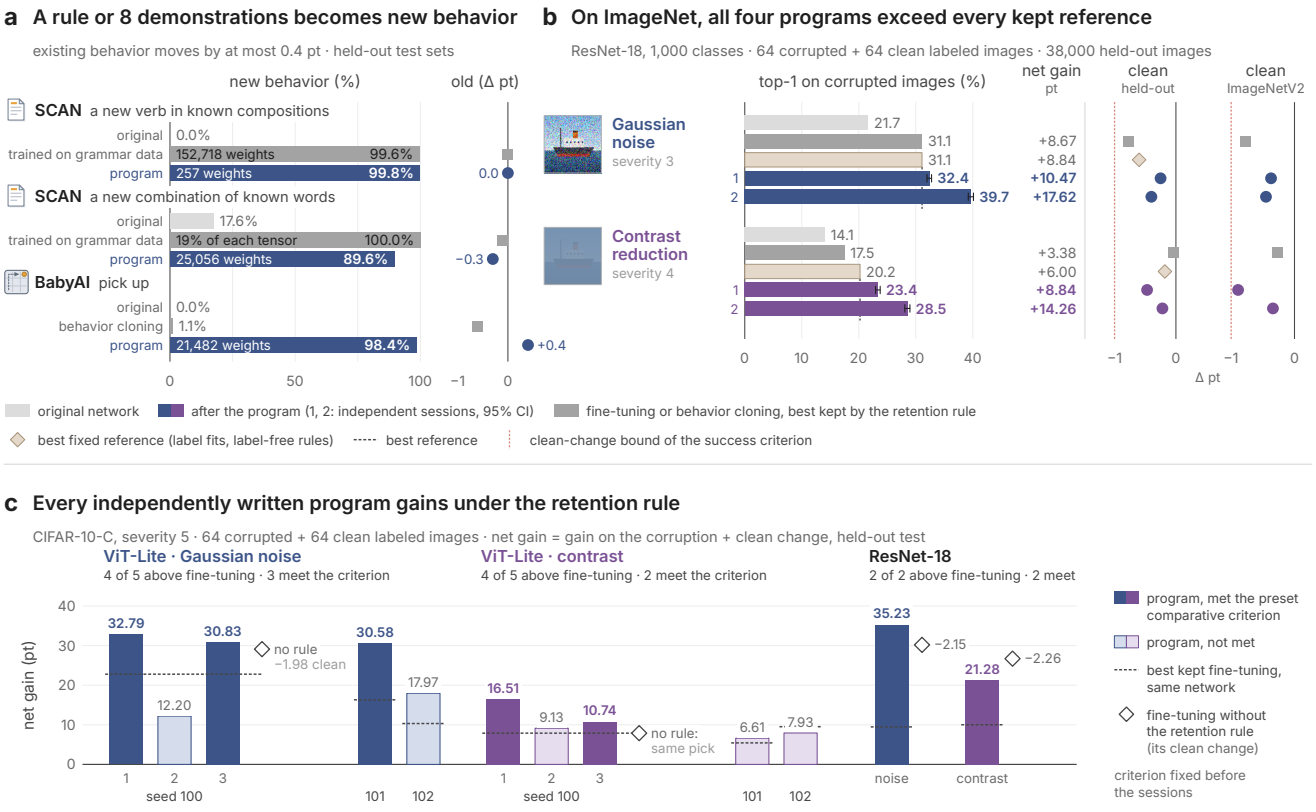


Figure 2. **Weight programs install new capabilities and keep existing behavior.** (a) SCAN (a new verb in known compositions; a new combination of known words) and BabyAI before and after the program, with the weights each changes; gray, a rule-to-data reference trained on commands generated from the grammar (SCAN) and the best behavior cloning kept by the retention rule (BabyAI). (b) ImageNet ResNet-18: top-1 under corruption (95% intervals), net gain, and the change in clean top-1 on held-out ImageNet and on ImageNetV2; dashed red, the clean bound of the success criterion. (c) Net gain of every CIFAR-10 program, 10 of 12 above the best kept fine-tuning on its network; light bars, programs that do not meet the prespecified comparative criterion; diamonds, fine-tuning without the retention rule. Held-out test sets.

from the plain mean difference of the noisy and clean examples keeps +31.13, and the same gate in two units drawn at random from the block gives +31.86 (Tab. 10). Gradient training of exactly the entries the noise and contrast programs change, from their original values with the fine-tuning grid and the same rule, reaches +3.60 and +0.72, and from the programs' values it keeps +33.35 and +18.00: the program's specific weight configuration, rather than the choice of edited entries alone, accounts for the gain. Network-specific information shapes the program: with access to the trained weights and outputs, the programs reach +31.75 and +17.98, against +10.78 and +11.18 when the same LLM receives the parameter names and shapes, the forward code, the examples and the checks, but no weight value or network output (one session per condition).

Different capabilities receive different computations. Unlike the noise program, the contrast program combines an input-wide adjustment of the patch-embedding filters with a condition-selective correction (Fig. 3d). It raises the zero-mean part of the filters by a factor of 1.45, a global gain on the first layer, and adds a gated rank-6 correction in eight freed units of block 2, whose gate, on its own, opens on 99.7% of low-contrast and 0.3% of clean patch tokens. On the original network the gain alone adds +10.95 points and the correction alone +9.63, at clean changes of 0.00 and +0.30, and together they reach +17.93: each part adds 7.0 and 8.3 points on top of the other. The noise computation contains no such first-layer gain; run as a template on contrast, the noise gate reaches +5.40 (Tab. 10). The two components also differ in reach: on the ViT-Lite and the CIFAR-10 ResNet-18 alike, the input-wide gain accounts for the changes on other corruptions, the gains under blur and fog and the losses under noise, whereas the gated correction acts selectively on contrast reduction (Fig. 9). Across architectures and target conditions, programs construct different forms of computation, including gates in little-used

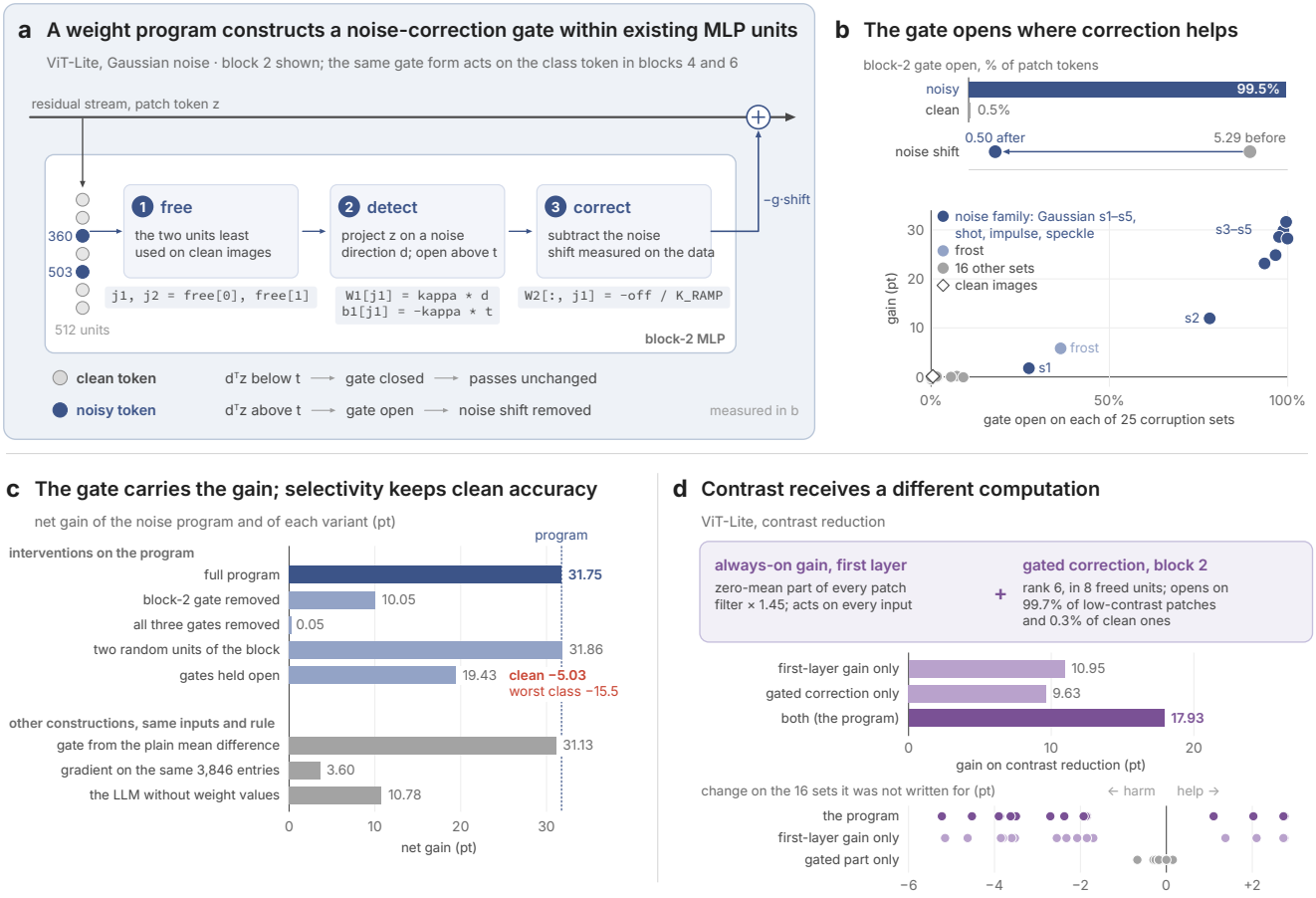


Figure 3. **Weight programs install identifiable, causally testable computations.** (a) The noise program constructs a gate within existing MLP units that removes the noise shift (schematic). (b) The gate opens on noisy, not clean, inputs, and its activation tracks the corruptions where the program helps. (c) Causal interventions and alternative constructions identify the gated computation, more than the choice of units, as the source of this program's gain (Tab. 10). (d) The contrast program combines an input-wide gain adjustment with a selective correction; the two components have distinct target-condition and off-target effects (Fig. 9). Selection split.

units, corrections of activation shifts, input-layer rescaling and convolutional filtering (Tab. 9). Across the ten ViT-Lite programs, those that meet the criterion repurpose little-used capacity into a condition-specific computation: gated corrections in freed MLP units, or a nearly unused attention head rewritten to subtract the image's mean color.

The BabyAI program writes a stated rule into the policy, a complementary form of mechanistic evidence. It copies the embeddings of *go* and *to* into *pick* and *up*, builds from dead units an exact detector for a faced cell of the named color and type that raises the pickup action, and adds a detector that stops the policy from pressing forward into a blocking object. An explicit rule reconstructed from these three parts predicts the edited policy's success or failure on all 2,000 evaluated episodes, 1,000 with the new mission and 1,000 with the old. The weights carry the stated rule, and the policy runs it with no outside code at the base network's inference cost.

4.3. Executable programs recompute effective edits across independently trained networks

When the same program is executed on five independently trained ViT-Lites, using each network's own 64 labeled examples, it constructs a network-specific edit without another LLM session. Rerun this way, the Gaussian-noise program gains +27.55 to +30.54 points (mean +28.57) at clean changes of −0.19 to +0.07, and the contrast program +16.04. All ten programs carry their gains over: on the five networks each was not written for, each keeps 83% to 110% of its own gain on the held-out test sets, and clean accuracy stays within 1 point on 48 of the 50 transfers (Fig. 4c). Reused the same way, the best fine-tuning recipe reaches +20.79 and +8.09 (Tab. 11).

Across independently trained ViT-Lites, rerunning a program transfers its effect more reliably than copying the source network's numerical edit (Fig. 4a,b). Added un-

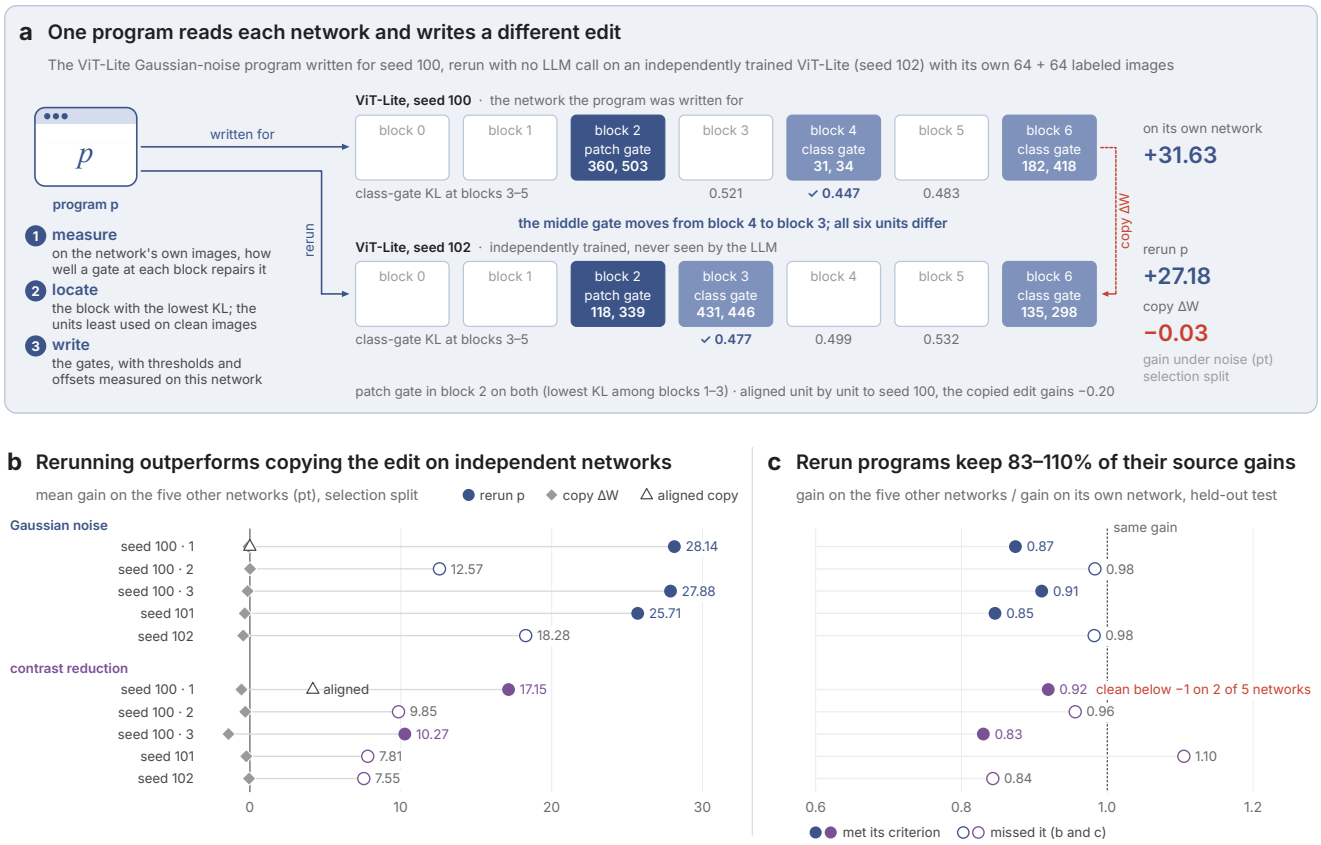


Figure 4. **Executable weight programs reconstruct effective edits across independently trained networks.** (a) Rerun on an independently trained ViT-Lite, the noise program places its gates anew and keeps its gain, while its fixed weight edit, copied unchanged, produces no gain on this network (recorded placements, schematic). (b) Ten programs, rerun or copied to five other networks (selection split). (c) Share of each program's own gain kept on the other networks (held-out test sets). For networks that share an initialization, see Fig. 10.

changed to the other ViT-Lites, the edits of the ten programs gain −1.42 to +0.01 points on average, 7.6 to 28.2 below rerunning the programs (selection split). A rerun recomputes where and how to write: on one network the noise program gates blocks 2, 3 and 6 instead of 2, 4 and 6, with thresholds measured on that network (Fig. 4a, Sec. A.8). Matching units does not recover this: aligned unit by unit to the original by activation matching on 2,048 clean images, the copied edits of the first program of each condition gain −0.01 and +4.17, whereas each rerun reconstructs its edit from the target network's own computation. Copied edits transfer where networks share coordinates: when ResNet-18s share a pretrained initialization, they keep 96% and 87% of the rerun gain (selection split, Fig. 10), while a program transfers across independently trained networks as well.

4.4. Successive programs extend edited networks and keep earlier gains

Executable weight programs install capabilities in succession. On the ViT-Lite, after the Gaussian-noise program, six contrast programs written in separate sessions for the edited network add +5.23 to +14.50 points of contrast robustness on the held-out test sets, while the installed noise robustness moves by −0.17 to +0.23 and clean accuracy stays within 1.47 points of the original network (Fig. 5a,c, Tab. 13); four exceed the 10-point threshold for the additional capability fixed before the sessions. In the reverse order, all six noise programs written for the contrast-programmed network add +30.9 to +37.7 points of noise robustness while contrast moves by −0.28 to +0.03 (selection split, Fig. 5a).

The earlier capability is kept in the computation itself. None of the six contrast programs writes the two units of the noise gate or blocks 4 and 6 (Fig. 5b): constructed from the edited network, the earlier program and checks on the earlier condition, they write around the installed gate.

A counterfactual rerun shows why the current state of the network matters. A contrast program written for the original network, applied after the noise program, frees the two block-2 units that hold the noise gate, because they are idle on clean images, and breaks 1,224 of 4,000 noisy check

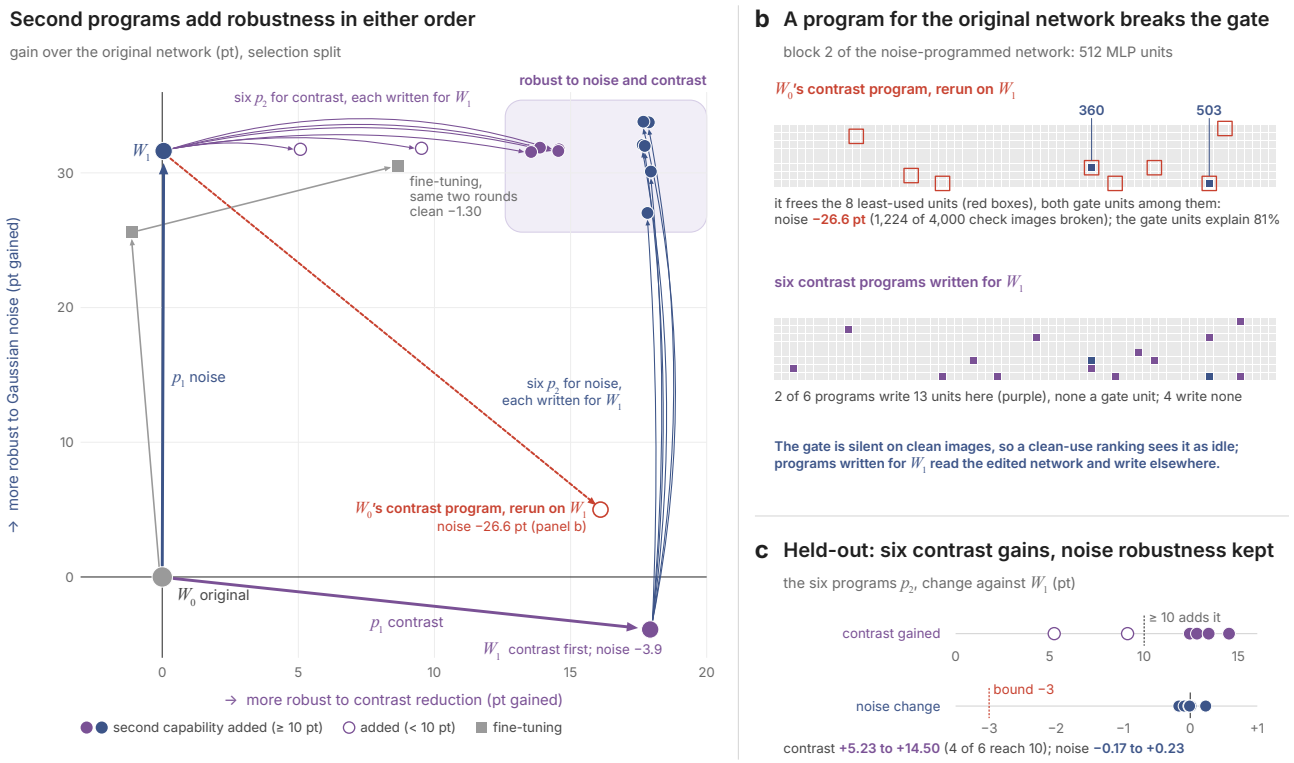


Figure 5. **Programs written for an edited network add a capability and keep the earlier one.** (a) Gains over the original network in both orders: six alternative second programs per order, each written for W_1; red, the W_0 program rerun on W_1; gray, fine-tuning through the same two rounds (selection split). (b) Why a program written for the original network disrupts the installed noise gate when rerun on W_1: the block-2 units each program writes. (c) Held-out test sets for the noise-first order: all six programs add contrast robustness, four by at least the 10-point threshold fixed in advance.

images; freeing those units accounts for 81% of its noise loss (selection split, Fig. 5a,b).

5. Discussion

The central finding is that a trained neural network can be extended by an executable construction procedure whose output becomes part of the network's own computation. The updated network performs the new capability on its own, while the program retains the procedure that built it from existing computation. Capability installation therefore has an explicit implementation that can be tested by intervention and recomputed by rerunning the program. This separates the reusable specification of a capability from the network-specific weight values that implement it.

These results reveal a mechanism of capability acquisition in which task knowledge is translated into new computation by recruiting and reorganizing operations already represented within a trained network. Rules, examples and checks specify the desired behavior, while the network's weights and activations inform how it can be implemented: programs reconnect existing representations, recruit learned operations or add corrections for particular inputs. In the noise-gating programs, preservation is built into the computation itself, in addition to the retention rule that selects them: the correction activates mainly on affected inputs and leaves clean representations nearly unchanged, and causal interventions separate the two roles, with the installed computation producing the robustness gain and its selective activation protecting clean accuracy. The source of a new capability and the reason the measured existing behavior is kept can thus be examined directly and used to inform the design of later programs.

Reusability changes what can be retained after an update: not only the modified network, but also an executable description of how the modification was constructed. Rerun on another network, that description recomputes an edit for the network's own weights; written for an edited network, a new program can account for what earlier programs installed. As capabilities accumulate, previously installed computations become part of the foundation that later updates can use and preserve, and these properties motivate reusable program libraries and longer sequences of capability updates, each retained together with its checks.

Natural next steps are programs that construct representations a network does not yet have (Sec. A.10) and programs that extend foundation models, including LLMs, with new procedures or behaviors after deployment. More broadly, executable weight programs provide a foundation for self-directed capability extension, in which proposed changes to a network's computation are evaluated independently, retained and built upon through further programs, each remaining code that can be read and tested.

Training establishes the initial computation of a neural network. Executable weight programs offer a complementary route: task knowledge becomes part of the network's own computation while its measured existing behavior is kept, and each update is retained as a procedure that can be inspected, rerun and extended. Weight programming thereby opens a path to cumulative capability acquisition, in which new computations are installed, reused and progressively extended within trained neural networks.

References

[1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2022. 2, 3, 5

[2] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018. 2, 3, 5

[3] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 2, 3

[4] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-Editing Memory in a Transformer. In *International Conference on Learning Representations (ICLR)*, 2023. 2, 3

[5] Steffen Schneider, Evgenia Rusak, Luisa Eck, Oliver Bringmann, Wieland Brendel, and Matthias Bethge. Improving robustness against common corruptions by covariate shift adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2, 3, 5

[6] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully Test-time Adaptation by Entropy Minimization. In *International Conference on Learning Representations (ICLR)*, 2021. 2, 3, 5, 20

[7] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. 2, 3

[8] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. 2, 3

[9] Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights. Blog post, https://rikiyat.github.io/blog/weight-space-programming/, 2026. Published November 2, 2026. 2, 3

[10] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019. 3

[11] Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit Inductive Bias for Transfer Learning with Convolutional Networks. In *International Conference on Machine Learning (ICML)*, 2018. 3, 5

[12] Michael McCloskey and Neal J. Cohen. *Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem*, pages 109–165. Elsevier, 1989. 3

[13] R French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999. 3

[14] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 3

[15] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On Tiny Episodic Memories in Continual Learning. *arXiv preprint arXiv:1902.10486*, 2019. 3

[16] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Zhiquan Wen, Yaofo Chen, Peilin Zhao, and Mingkui Tan. Towards Stable Test-Time Adaptation in Dynamic Wild World. In *International Conference on Learning Representations (ICLR)*, 2023. 3, 5, 20

[17] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *International Conference on Machine Learning (ICML)*, 2020. 3

[18] Anton Sinitsin, Vsevolod Plokhotnyuk, Dmitriy Pyrkin, Sergei Popov, and Artem Babenko. Editable Neural Networks. In *International Conference on Learning Representations (ICLR)*, 2020. 3

[19] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast Model Editing at Scale. In *International Conference on Learning Representations (ICLR)*, 2022. 3

[20] David Bau, Steven Liu, Tongzhou Wang, Jun-Yan Zhu, and Antonio Torralba. *Rewriting a Deep Generative Model*, pages 351–369. Springer International Publishing, 2020. 3

[21] Shibani Santurkar, Dimitris Tsipras, Mahalaxmi Elango, David Bau, Antonio Torralba, and Aleksander Madry. Editing a classifier by rewriting its prediction rules. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 3

[22] Thomas Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 3

[23] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing Models with Task Arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[24] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving Interference When Merging Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 3

[25] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging Models modulo Permutation Symmetries. In *International Conference on Learning Representations (ICLR)*, 2023. 3, 4

[26] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2024. 3

[27] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research (TMLR)*, 2024.

[29] Chris Lu, Samuel Holt, Claudio Fanconi, Alex J. Chan, Jakob Foerster, Mihaela van der Schaar, and Robert Tjarko Lange. Discovering Preference Optimization Algorithms with and for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 3

[30] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 3

[31] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. 3

[32] David Ha, Andrew Dai, and Quoc V. Le. HyperNetworks. In *International Conference on Learning Representations (ICLR)*, 2017. 3

[33] Aviv Navon, Aviv Shamsian, Idan Achituve, Ethan Fetaya, Gal Chechik, and Haggai Maron. Equivariant Architectures for Learning in Deep Weight Spaces. In *International Conference on Machine Learning (ICML)*, 2023.

[34] Allan Zhou, Kaien Yang, Kaylee Burns, Adriano Cardace, Yiding Jiang, Samuel Sokota, J. Zico Kolter, and Chelsea Finn. Permutation Equivariant Neural Functionals. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[35] Konstantin Schürholt, Dimche Kostadinov, and Damian Borth. Hyper-Representations: Self-Supervised Representation Learning on Neural Network Weights for Model Characteristic Prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[36] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking Like Transformers. In *International Conference on Machine Learning (ICML)*, 2021.

[37] David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled Transformers as a Laboratory for Interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[38] Dan Friedman, Alexander Wettig, and Danqi Chen. Learning Transformer Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 3

[39] Mengran Li, Bo Li, Weijing Zhao, Chengyang Zhang, Boyu Zhang, Jinfeng Xu, Zelin Zang, Victor C. M. Leung, and Stan Z. Li. WeightLoop: Weight-Space Programming with Experimental Feedback. *arXiv preprint arXiv:2611.07319*, 2026. 3

[40] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*, 2021. 4

[41] Ali Hassani, Steven Walton, Nikhil Shah, Abulikemu Abuduweili, Jiachen Li, and Humphrey Shi. Escaping the Big Data Paradigm with Compact Transformers. *arXiv preprint arXiv:2104.05704*, 2021. 4

[42] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016. 4

[43] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3): 211–252, 2015. 4

[44] Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International Conference on Machine Learning (ICML)*, 2018. 4, 5

[45] João Loula, Marco Baroni, and Brenden Lake. Rearranging the familiar: Testing compositional generalization in recur-

rent networks. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 108–114. Association for Computational Linguistics, 2018. 4, 5

[46] Dan Hendrycks and Thomas Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. 4

[47] Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning. In *International Conference on Learning Representations (ICLR)*, 2019. 5

[48] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet Classifiers Generalize to ImageNet? In *International Conference on Machine Learning (ICML)*, pages 5389–5400, 2019. 6, 19

[49] Chi Zhang, Nan Song, Guosheng Lin, Yun Zheng, Pan Pan, and Yinghui Xu. Few-shot incremental learning with continually evolved classifiers. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12450–12459. IEEE, 2021. 23

A. Supplementary material

This appendix provides implementation details, complete session-level results, additional controls, cross-condition analyses and exploratory experiments beyond the main protocol, in the order of Sec. 4. Tables 5, 6, 12 and 15 repeat Tabs. 3, 4, 11 and 14 with the numbers of the selection split.

A.1. Protocol and implementation

The protocol specifies the task knowledge, the evaluation feedback and the retention bound for each setting (Tab. 1).

What the protocol admits. Under the main protocol, weight updates are constructed from forward evaluations, network statistics and closed-form computations. Gradient-based and finite-difference optimization of objectives computed from the network's outputs (by automatic differentiation, hand-written backpropagation or finite differences), solvers on label-derived targets and iterated label-driven updates are excluded. Three practices fall within the protocol: one overall edit strength found by a search against a label-free bound on the change of clean outputs, a one-shot closed-form bias set from the network's own logit margins (on BabyAI, where the only label-derived quantity is the index of the pickup action the task names), and a head bias set so that the edited network's mean logits on label-free inputs equal the original network's. Manual code inspection identified three exploratory sessions that size their edits from finite differences of label-free output quantities: the ViT-Lite session given only the noise formula, and rounds 1 and 3 of a ResNet-20 stream. They fall outside the main protocol, are reported in Sec. A.11, and are not used to support the claims of Sec. 4.

Sessions and sandbox. The following measurements describe the cost of constructing a program, distinct from the cost of rerunning a kept program. The LLM works in a Python kernel of at most 150 cells, within 90 to 120 minutes (140 on ImageNet and 190 on the CIFAR-10 ResNet-18, whose checks take longer). A session takes 30 to 130 minutes, and the CIFAR-10, SCAN and BabyAI sessions cost US$5 to 13 at API prices. Rerunning a kept program on a further network takes a few minutes with no LLM call. A program runs in a sandbox with NumPy only, no network access and a 300 s limit. It may change at most 20% of the weights, and the change to each tensor of rank two or more is at most half of that tensor's Frobenius norm. The per-tensor limit applies to the image classifiers but not to the SCAN Transformers or the BabyAI policy, whose edits write whole embedding or output rows.

Labels and computation in a session. The 8 external checks count labeled check images at every level of information, including the levels with condition labels withheld and with the name only. A single-round session scores at most 8 candidate weight edits against the labeled examples in I, as each fine-tuning method tries 8 configurations. Two kinds of label-scored work are not counted toward this cap: predictors that change no weight of the network, such as output offsets or nearest-class-mean probes on its features, and selection a program makes inside its own run. This counting rule was set on 2026-11-06, after two sessions had been read: the noise-formula session (Sec. A.11), which scores 5 candidate edits under the rule and 9 settings with its four logit offsets, and the CIFAR-10 ResNet-18 contrast session, which scores 6 under the rule and 9 with its three nearest-class-mean probes. The rule applies to every session, and the automatic audit counts both kinds, so its counts are recorded beside the hand counts. The cap applies to single-round sessions. A session that writes a later round scores 0 to 44 settings against that round's labeled examples, including threshold sweeps, and the fine-tuning streams fit the same labels. On SCAN, whose grammar gives the answer to any command, the LLM may test candidates against answers it derives from the rule. A kept program computes from the network it runs on each time it runs, with forward passes, activation statistics, SVD and PCA on the inputs in I. The contrast program written without weight values (Sec. 4.2) also selects among eight configurations by cross-validation on the labeled examples each time it runs. The audit flags gradients, solvers on label-derived targets, iterated label-driven updates and the number of settings scored against labels.

Data and networks. Corruptions are generated with the official CIFAR-10-C code on held-out CIFAR-10 training images. On the clean test images this code reproduces the official files exactly for defocus and contrast and within 1.2% of their median pixel deviation for noise and snow. The ViT-Lite follows the authors' CIFAR-10 recipe. Gaussian noise leaves it 35.55% accuracy and contrast reduction 58.88%, while defocus and snow cost it 12 and 13 points. The ResNet-18 receives a new 10-way head and is fine-tuned in full with inputs upsampled to 128×128. The BabyAI policy is a feedforward network with FiLM conditioning on the instruction, trained by imitation on 50,000 expert demonstrations of the go-to mission in one room. Each SCAN Transformer (width 128, 4 layers) is trained on the official training commands of its split except 3,000 held out for the old checks, selection and test commands. The reuse networks are trained from seeds 101 to 105, and from seed 100 for programs written on seeds 101 and 102.

A.2. Comparison with fine-tuning

This section evaluates how the relative performance of weight programs and fine-tuning changes with the data bud-

Network and new condition	Task knowledge I given to the LLM	Checks and retention rule	References	Selection and test sets
ViT-Lite and ResNet-18; Gaussian noise or contrast reduction (CIFAR-10-C, severity 5)	64 labeled images of the condition (16 the network misclassifies, 48 at random) and 64 labeled clean images, with the network's predictions; on the ViT-Lite also the lower levels of Sec. A.5	S_{new} and S_{old} on 4,000 check images; $S_{\text{old}} \geq -20$	eight fine-tuning methods, 8 configurations each; eleven fixed references (label fits, Tent, BN adaptation, SAR)	4,000 images per condition; official CIFAR-10 and CIFAR-10-C test files
Successive updates: ResNet-20 with Gaussian noise, defocus, snow and contrast in turn; ViT-Lite with Gaussian noise and contrast in either order	per round, 64 labeled images of the new condition and the code of the earlier programs; in one ViT-Lite arm also the list of entries round 1 changed	checks report every condition so far; stream clean change ≥ -60 of 4,000, each earlier condition ≥ -20 per round	ResNet-20: four fine-tuning streams re-selecting among 8 configurations each round, and fixed reference streams; ViT-Lite: criteria fixed in advance (Sec. A.9)	as above
SCAN Transformer; add primitive (*jump*) or around right	the interpretation grammar, 64 old commands with answers and the network's predictions, and on add primitive the training row of *jump*	S_{new} on a third of the official test commands, S_{old} on 1,000 old commands; $S_{\text{old}} \geq -5$	fine-tuning on the given rows; fine-tuning on commands generated from the grammar	a second third and 1,000 old commands; the last third and 1,000 old commands
BabyAI policy; pick up	8 demonstrations of the new mission (observation, instruction and expert action at each step) and 64 old steps, with the policy's action logits	rollouts of 500 new-mission and 500 old-mission episodes; $S_{\text{old}} \geq -3$	the eight fine-tuning methods and behavior cloning of parts of the policy; full behavior cloning from 16 to 256 demonstrations	1,000 episodes per mission; 1,000 further episodes per mission
ResNet-18 on ImageNet (1,000 classes); Gaussian noise (severity 3) or contrast reduction (severity 4) of ImageNet-C	as for CIFAR-10, from held-out validation images	as for CIFAR-10, on 4,000 validation images each	eight fine-tuning methods; eleven fixed references (label fits, Tent, BN adaptation)	4,000 validation images per condition; the other 38,000 validation images

Table 1. **What each task gives the LLM and the references.** A session is one LLM session with 8 checks per round and the retention rule of Sec. 3.1 with the bound shown, and the network is left unchanged when no candidate meets it. S_{new} and S_{old} count inputs fixed minus inputs broken, on the new condition and on old inputs. Every reference receives the same examples or rule and the same checks and is kept by the same rule. Matched checks are the external evaluation checks; label-scored work inside a session is counted as described in Sec. A.1. Analyses are read on the selection set, and each kept network is read once on the test set.

get, the information about the corruption and the permitted loss of clean accuracy. On the ViT-Lite, no fine-tuning variant reaches the program's net gain within the retention rule; among the evaluated variants, matching or exceeding it requires the true corruption model and no clean bound, or 64 times the labeled images or more (Tab. 4, Fig. 7, Fig. 6). On BabyAI, no configuration of the eight fine-tuning methods keeps an edit within the retention rule, and full behavior cloning keeps no edit even from 256 demonstrations (selection split).

Same examples and checks. Under the matched-data and retention protocol, the ViT-Lite programs reach net gains of +32.79 and +16.51, against +22.78 and +7.89 for the best of the eight evaluated methods. Tuning the recipe with warmup, augmentation and minibatches keeps +21.34 and +7.58.

A corruption model. Images synthesized from a corruption model estimated from the 64 images add no independently labeled observations, although they carry the assumptions of the fitted model. Fine-tuning with them keeps no edit under Gaussian noise and reaches +10.00 under contrast reduction. The true corruption model adds the process that generates the condition, which the programs are not given. With it, fine-tuning keeps no edit under Gaussian noise and reaches +10.23 under contrast reduction.

Trade-offs under relaxed bounds. Relaxing the clean-accuracy bound changes the attainable trade-off between gain and retention: weight programs hold their largest advantage under tight bounds, and the evaluated fine-tuning configurations reach larger gains when larger clean-accuracy changes are permitted. Without the rule, its best configurations reach +29.11, +30.18 and +26.70 on the test sets at clean changes of -1.98 to -2.26, the last above

		Weight program (WeightCode)				Other route from the same information		
Setting	Given	new, base	new, after	old	weights	route	new	old
SCAN, add primitive	grammar, 64 old commands, the *jump* row	0.0%	99.8%	0.0	257	grammar-generated fine-tuning	99.6%	0.0
SCAN, around right	grammar, 64 old commands	17.6%	89.6%	−0.3	25,056	grammar-generated fine-tuning	100.0%	−0.1
BabyAI, pick up	8 demonstrations, 64 old steps	0.0%	98.4%	+0.4	21,482	behavior cloning, embeddings and head	1.1%	−0.6

Table 2. **Programs enable behaviors the original networks did not perform.** New: exact match on the SCAN commands that contain the new primitive or the new combination, or success on the new BabyAI mission, before and after the update. Old: change in points on the old commands or the old mission. Weights: entries the program changes. Grammar-generated fine-tuning trains on commands generated from the grammar and mixed with the old ones, and changes 152,718 entries on add primitive and a fixed 19% of each tensor on around right; its generated commands include some test commands, so it is a rule-to-data reference rather than a held-out supervised baseline. Behavior cloning of the embeddings and the head is the strongest reference kept by the program's rule on BabyAI. Fine-tuning on the given rows alone is in Sec. 4.1. Held-out test sets.

		Weight program (WeightCode)			Fine-tuning, best of 8		Fixed refs.	Weights
Network	Condition	gain	clean	net gain	kept	no bound	kept	changed
ImageNet, 1,000 classes, corruptions of ImageNet-C at severity 3 (noise) and 4 (contrast)								
ResNet-18	Gaussian noise	+10.72	−0.25	**+10.47**	+8.67	–	+8.84	2,152,928
ResNet-18	contrast reduction	+9.31	−0.47	**+8.84**	+3.38	–	+6.00	33,176
CIFAR-10, corruptions of CIFAR-10-C at severity 5								
ViT-Lite	Gaussian noise	+32.68	+0.11	**+32.79**	+22.78	+29.11 (−1.98)	+16.48	3,846
ViT-Lite	contrast reduction	+17.45	−0.94	**+16.51**	+7.89	+7.89 (−0.34)	+8.25	16,648
ResNet-18	Gaussian noise	+35.61	−0.38	**+35.23**	+9.47	+30.18 (−2.15)	+9.22	1,718,057
ResNet-18	contrast reduction	+21.43	−0.15	**+21.28**	+10.01	+26.70 (−2.26)	+13.51	37,192

Table 3. **Under matched task data, checks and clean-accuracy retention, weight programs achieve larger net gains than the evaluated fine-tuning methods.** Accuracy changes in points on the held-out test sets. ImageNet: torchvision's ResNet-18 with its 1,000 classes (Sec. A.4). CIFAR-10: ViT-Lite-7/4 trained on CIFAR-10 and an ImageNet-pretrained ResNet-18 fine-tuned on CIFAR-10. Net gain is the gain on the new condition plus the clean change. Every method receives the same 64 labeled images of the condition and 64 labeled clean images (the fine-tuning variant without clean images fits only the 64 of the condition) and the same 8 checks. Kept: the checked configuration with the best check score among those losing at most 0.5% of clean check images. No bound: the configuration with the best check score at any clean loss, with its clean change in brackets (not run on ImageNet). Fixed refs.: the best of eleven label fits and label-free rules, kept by the same rule. Weights: entries the program changes. Each program row is the first session on its network; every ImageNet session is in Tab. 8 and every ViT-Lite session in Tab. 7. Bold: the highest net gain among the methods kept by the rule.

the CIFAR-10 ResNet-18 contrast program (Tab. 3); when the clean-loss bound allows 2 points, fine-tuning passes the program by 0.78 points under Gaussian noise on the ViT-Lite and by 4.95 under contrast reduction on the CIFAR-10 ResNet-18, at clean changes of −1.58 and −2.38 (selection split). On the ViT-Lite under Gaussian noise, the tuned recipe reaches +31.13 at −2.23 and the true corruption model +32.97 at −2.78. Under contrast reduction the best variant reaches +11.49. Figure 7b–e shows every checked configuration on the selection split.

More labeled images. On the ViT-Lite, with N labeled images of the condition and as many clean ones, the evaluated full fine-tuning recipe first reaches the program's net gain at $N = 4{,}096$ under Gaussian noise, 64 times the program's 64, and between $N = 4{,}096$ and $40{,}000$ under contrast reduction (Fig. 6, selection split). Under noise the retention rule sets this amount: at $N = 64$ to $1{,}024$ the same fine-tunes gain +30.7 to +38.9 but lose more clean check images than the rule allows.

A.3. Every session

Table 7 reports every independent ViT-Lite session, so that installation, the comparison with fine-tuning and reuse can be assessed separately. Of the ten programs, eight exceed the best kept fine-tuning on their own network, five meet the prespecified success criterion, and all ten keep 0.83 to 1.10 of their gain when rerun on the other five networks. Un-

		Gaussian noise			Contrast reduction		
		gain	clean	net gain	gain	clean	net gain
Weight program (WeightCode)		**+32.68**	**+0.11**	**+32.79**	**+17.45**	−0.94	**+16.51**
Fine-tuning, kept by the same rule	best of 8 methods	+23.41	−0.63	+22.78	+8.23	−0.34	+7.89
	best tuned recipe	+21.76	−0.42	+21.34	+7.90	**−0.32**	+7.58
	with an estimated corruption model	unchanged network			+10.83	−0.83	+10.00
	with the true corruption model	unchanged network			+11.00	−0.77	+10.23
Fine-tuning, no clean-loss bound	best of 8 methods	+31.09	−1.98	+29.11	+8.23	−0.34	+7.89
	best tuned recipe	+33.36	−2.23	+31.13	+7.90	−0.32	+7.58
	with the true corruption model	+35.75	−2.78	+32.97	+12.39	−0.90	+11.49
Best of 11 fixed references, kept				+16.48			+8.25

Table 4. **Fine-tuning on the ViT-Lite under matched data, with and without the clean-accuracy retention bound.** ViT-Lite-7/4 on CIFAR-10-C at severity 5, accuracy changes in points on the held-out test sets. Every method sees the same 64 labeled corrupted and 64 labeled clean images and receives 8 checks. "Kept by the same rule" is the checked configuration with the best check score among those losing at most 0.5% of clean check images, and "unchanged network" means that no configuration met the bound. Best of 8 methods: LoRA r4 (Gaussian) and full fine-tuning with L2-SP (contrast), and without the bound LoRA r16 and L2-SP. Tuned recipes add warmup, augmentation and minibatches. The corruption-model rows add images synthesized from a corruption model estimated from the 64 images, which adds no information, or from the true one, which adds the process that generates the condition. Bold: the best value of each column among the methods kept by the rule.

		Weight program (WeightCode)			Fine-tuning, best of 8		Fixed refs.	Weights
Network	Condition	gain	clean	net gain	kept	no bound	kept	changed
ImageNet, 1,000 classes, corruptions of ImageNet-C at severity 3 (noise) and 4 (contrast)								
ResNet-18	Gaussian noise	+10.23	−0.10	**+10.13**	+8.70	–	+8.00	2,152,928
ResNet-18	contrast reduction	+9.68	−1.13	**+8.55**	+3.40	–	+5.33	33,176
CIFAR-10, corruptions of CIFAR-10-C at severity 5								
ViT-Lite	Gaussian noise	+31.62	+0.12	**+31.75**	+22.90	+30.50 (−1.43)	+16.08	3,846
ViT-Lite	contrast reduction	+17.93	+0.05	**+17.98**	+8.65	+8.65 (0.00)	+9.48	16,648
ResNet-18	Gaussian noise	+34.98	−0.38	**+34.60**	+9.15	+29.25 (−2.38)	+9.10	1,718,057
ResNet-18	contrast reduction	+21.48	−0.33	**+21.15**	+9.78	+26.13 (−2.80)	+13.45	37,192

Table 5. **Table 3 on the selection split.** Accuracy changes in points on the selection split. ImageNet: torchvision's ResNet-18 with its 1,000 classes (Sec. A.4). CIFAR-10: ViT-Lite-7/4 trained on CIFAR-10 and an ImageNet-pretrained ResNet-18 fine-tuned on CIFAR-10. Net gain is the gain on the new condition plus the clean change. Every method receives the same 64 labeled images of the condition and 64 labeled clean images (the fine-tuning variant without clean images fits only the 64 of the condition) and the same 8 checks. Kept: the checked configuration with the best check score among those losing at most 0.5% of clean check images. No bound: the configuration with the best check score at any clean loss, with its clean change in brackets (not run on ImageNet). Fixed refs.: the best of eleven label fits and label-free rules, kept by the same rule. Weights: entries the program changes. Each program row is the first session on its network; every ImageNet session is in Tab. 8 and every ViT-Lite session in Tab. 7. Bold: the highest net gain among the methods kept by the rule.

der Gaussian noise three of five sessions meet the criterion, and under contrast reduction two of five. On the seed-100 network every contrast program exceeds the best kept fine-tuning (+9.13 to +16.51 against +7.89), and two of three noise programs do (+30.83 and +32.79 against +22.78). For the first Gaussian-noise session on the seed-100 network, an expanded reference grid was fixed after the session began but before its first check; the program meets both the original and the expanded criterion.

Across these sessions, higher net gains are associated with the computation the program writes. The three Gaussian-noise programs that meet the criterion build a saturating gate from MLP units with little effect on clean outputs; the other two apply Wiener shrinkage without an MLP gate. Every contrast program raises the first-layer filters. The two that meet the criterion also turn little-used capacity into a correction, gated in freed MLP units or written into a block-0 attention head that subtracts the image's mean color. The three that add no such correction gain +6.77 to +9.51. Rerun on the other five networks, every program keeps clean accuracy within 1 point except the first contrast program, which loses more on two of five. Every second-

		Gaussian noise			Contrast reduction		
		gain	clean	net gain	gain	clean	net gain
Weight program (WeightCode)		**+31.62**	**+0.12**	**+31.75**	**+17.93**	+0.05	**+17.98**
Fine-tuning, kept by the same rule	best of 8 methods	+23.52	−0.62	+22.90	+8.65	+0.00	+8.65
	best tuned recipe	+22.32	−0.43	+21.90	+8.62	**+0.07**	+8.70
	with an estimated corruption model	unchanged network			+11.58	−0.18	+11.40
	with the true corruption model	unchanged network			+11.85	−0.15	+11.70
Fine-tuning, no clean-loss bound	best of 8 methods	+31.92	−1.43	+30.50	+8.65	+0.00	+8.65
	best tuned recipe	+34.10	−1.57	+32.52	+8.62	+0.07	+8.70
	with the true corruption model	+35.00	−2.43	+32.58	+13.30	−0.18	+13.12
Best of 11 fixed references, kept				+16.08			+9.48

Table 6. **Table 4 on the selection split.** ViT-Lite-7/4 on CIFAR-10-C at severity 5, accuracy changes in points on the selection split. Every method sees the same 64 labeled corrupted and 64 labeled clean images and receives 8 checks. "Kept by the same rule" is the checked configuration with the best check score among those losing at most 0.5% of clean check images, and "unchanged network" means that no configuration met the bound. Best of 8 methods: LoRA r4 (Gaussian) and full fine-tuning with L2-SP (contrast), and without the bound LoRA r16 and L2-SP. Tuned recipes add warmup, augmentation and minibatches. The corruption-model rows add images synthesized from a corruption model estimated from the 64 images, which adds no information, or from the true one, which adds the process that generates the condition. Bold: the best value of each column among the methods kept by the rule.

			On its own network							Rerun on the five other networks		
Condition	Network	Session	gain	clean	net	FT kept	criterion	checks	US$	share	clean ≥ -1	copied
Gaussian	seed 100	1	+32.68	+0.11	+32.79	+22.78	met	6	9.92	0.87	5 of 5	$-0.10^{\dagger}$
	seed 100	2	+12.73	−0.53	+12.20	+22.78	missed	8	8.36	0.98	5 of 5	$+0.01^{\dagger}$
	seed 100	3	+30.80	+0.03	+30.83	+22.78	met	8	8.75	0.91	5 of 5	$-0.16^{\dagger}$
	seed 101		+30.85	−0.27	+30.58	+16.31	met	5	7.81	0.85	5 of 5	$-0.34^{\dagger}$
	seed 102		+18.35	−0.38	+17.97	+10.32	missed	6	8.74	0.98	5 of 5	$-0.45^{\dagger}$
contrast	seed 100	1	+17.45	−0.94	+16.51	+7.89	met	8	9.63	0.92	3 of 5	$-0.56^{\dagger}$
	seed 100	2	+9.51	−0.38	+9.13	+7.89	missed	6	8.90	0.96	5 of 5	$-0.31^{\dagger}$
	seed 100	3	+11.39	−0.65	+10.74	+7.89	met	8	13.08	0.83	5 of 5	$-1.42^{\dagger}$
	seed 101		+6.77	−0.16	+6.61	+5.43	missed	8	10.33	1.10	5 of 5	$-0.24^{\dagger}$
	seed 102		+8.45	−0.52	+7.93	+9.52	missed	5	7.12	0.84	5 of 5	$-0.06^{\dagger}$

Table 7. **Every editing session on the ViT-Lite.** Each row is one LLM session under the protocol of Sec. 3 (8 checks, at most 120 minutes), on the seed-100 network or on networks trained from seeds 101 and 102; session 1 on the seed-100 network is the program of Tab. 3. Gain, clean and net gain in points on the held-out test sets. FT kept: the best fine-tuning configuration kept by the same rule on the same network. Criterion: the prespecified performance criterion of the comparative evaluation, fixed before the sessions (gain at least 10, clean change at least −1.0, net gain one point above the strongest reference within the program limits). Checks used and API cost per session. Rerun: the session's program run unchanged on the five other networks of its condition. Share: mean gain there over the gain on its own network; clean ≥ -1: networks whose clean change stays at or above −1.0; copied: mean gain when the program's edit is added unchanged instead (†: selection split).

round session is in Tab. 13. Each session used 5 to 8 checks and cost US$7.12 to 13.08.

A.4. ImageNet

On torchvision's ResNet-18 with its 1,000 ImageNet classes, all four programs meet the prespecified criterion on the held-out test set and exceed every fine-tuning method; one contrast session exceeds the clean-change bound on the selection split (Tab. 8).

Setup. The network is torchvision's IMAGENET1K_V1 ResNet-18, unchanged. The conditions are ImageNet-C's Gaussian noise and contrast functions, run as published on the 224×224 crops. The evaluated severities were selected with the original network on a separate split of 1,000 validation images that the test set excludes: severity 3 for Gaussian noise and 4 for contrast reduction, the highest severities that cost at least 15 points and keep at least 17% of the accuracy above chance (at severity 5 the network keeps 1.4% and 3.6%). Each session follows the protocol of the CIFAR-10 sessions, with the same 64 labeled images of each kind,

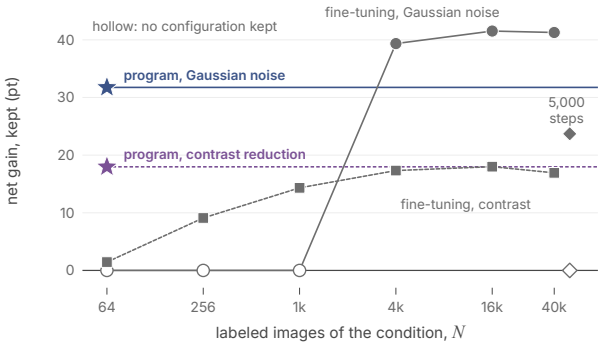


Figure 6. **Under Gaussian noise on the ViT-Lite, full fine-tuning needs substantially more labeled images to match the program under the same retention rule.** Net gain (points) of full fine-tuning on the ViT-Lite with N labeled images of the condition and N labeled clean images, kept by the same rule as the program (hollow: no configuration kept, so the network is unchanged), against the programs written from 64 images of the condition (stars and lines). Diamonds: 5,000 steps instead of at most 1,000, at $N = 40{,}000$. Selection split.

8 checks on 4,000 condition and 4,000 clean check images and the bound of 20 clean check images. The references are the eight fine-tuning methods and eleven fixed references of the CIFAR-10 ResNet-18. Their grids were extended once toward smaller steps, by a rule fixed before any session, since larger steps cost this network more clean accuracy than the bound allows.

Results. Under Gaussian noise the two programs reach net gains of +10.47 and +17.62 on the test set, against +8.67 for the best fine-tuning, and under contrast reduction +8.84 and +14.26, against +3.38. One contrast session has a clean change of −1.13 points on the selection split, on which the kept programs were read before the test set, and −0.47 on the test set; the other three programs meet the criterion on both splits. Full fine-tuning on 31 times the labeled images, 2,000 of each kind, keeps no configuration within the bound; its best configurations reach +20.53 and +30.60 at clean changes of −1.10 and −1.95 (selection split).

Neighboring severities and ImageNetV2. On 20,000 test images each program gains most at its own severity and keeps a positive gain at the neighboring ones. The largest change on the other condition is −3.36 points, for the first contrast program under Gaussian noise. On ImageNetV2 [48], 10,000 new images collected like the validation set, the programs change clean accuracy by −0.34 to −0.89 points and the kept fine-tuning by −0.77 and −0.27.

	Test set			Selection	ImageNetV2
	gain	clean	net gain	net gain	clean
Gaussian noise, severity 3; criterion: gain ≥ 7.0, net gain ≥ +9.84					
Program, session 1	+10.72	−0.25	+10.47	+10.13	−0.37
Program, session 2	+18.02	−0.40	+17.62	+16.38	−0.45
Fine-tuning, kept	+9.45	−0.78	+8.67	+8.70	−0.77
Best fixed reference	+9.44	−0.60	+8.84	+8.00	–
BN adaptation	+1.22	−0.14	+1.08	+0.93	−0.33
Contrast reduction, severity 4; criterion: gain ≥ 8.5, net gain ≥ +7.00					
Program, session 1	+9.31	−0.47	+8.84	+8.55*	−0.89
Program, session 2	+14.48	−0.22	+14.26	+15.83	−0.34
Fine-tuning, kept	+3.42	−0.04	+3.38	+3.40	−0.27
Best fixed reference	+6.18	−0.18	+6.00	+5.33	–

Table 8. **On ImageNet, all four programs meet the success criterion and exceed fine-tuning.** The ResNet-18 of torchvision with its 1,000 classes, two sessions per condition. Accuracy changes in points. Test set: 38,000 held-out ImageNet validation images, read once; the paired 95% intervals of the programs' values are within ±0.49 points. Selection: 4,000 images. Criterion on the test set: gain at least the bound shown, clean change at least −1.0 and net gain at least one point above the best fixed reference on the same images. *On the selection split this program's clean change is −1.13, past the −1.0 bound. Fine-tuning, kept: the best of the eight methods on the selection split, BatchNorm only under noise and full under contrast. BN adaptation re-estimates the BatchNorm statistics from the 64 condition images without labels; under contrast it keeps the original network. ImageNetV2: change in top-1 accuracy on the 10,000 ImageNetV2 matched-frequency images (original network 57.28%), paired 95% intervals within ±0.43 points.

What the programs write. The Gaussian-noise programs estimate the noise from pixel statistics of the given images, synthesize noisy copies of the clean ones and remove the noisy-minus-clean shift of activations in the deep half of the network: the first at the inputs of two residual stages and of the classifier, folded into their weights and BatchNorm statistics, and the second as a rank-one term in eight convolutions that a whitened noise detector switches on.

The contrast programs measure the contrast factor from pixel statistics and turn the two least-used units of several residual blocks into gates that open on low-contrast inputs and add the activation shift that restores them. The first gate opens from the 75th percentile of the clean projections, so a quarter of clean locations receive part of the correction, and the second only above the 99.5th, which separates their clean costs.

A.5. Less information

On the ViT-Lite under Gaussian noise, a program meets the bar fixed before these sessions from labeled examples and with the labels of the condition images withheld (Fig. 8). Each information condition is evaluated in one session on

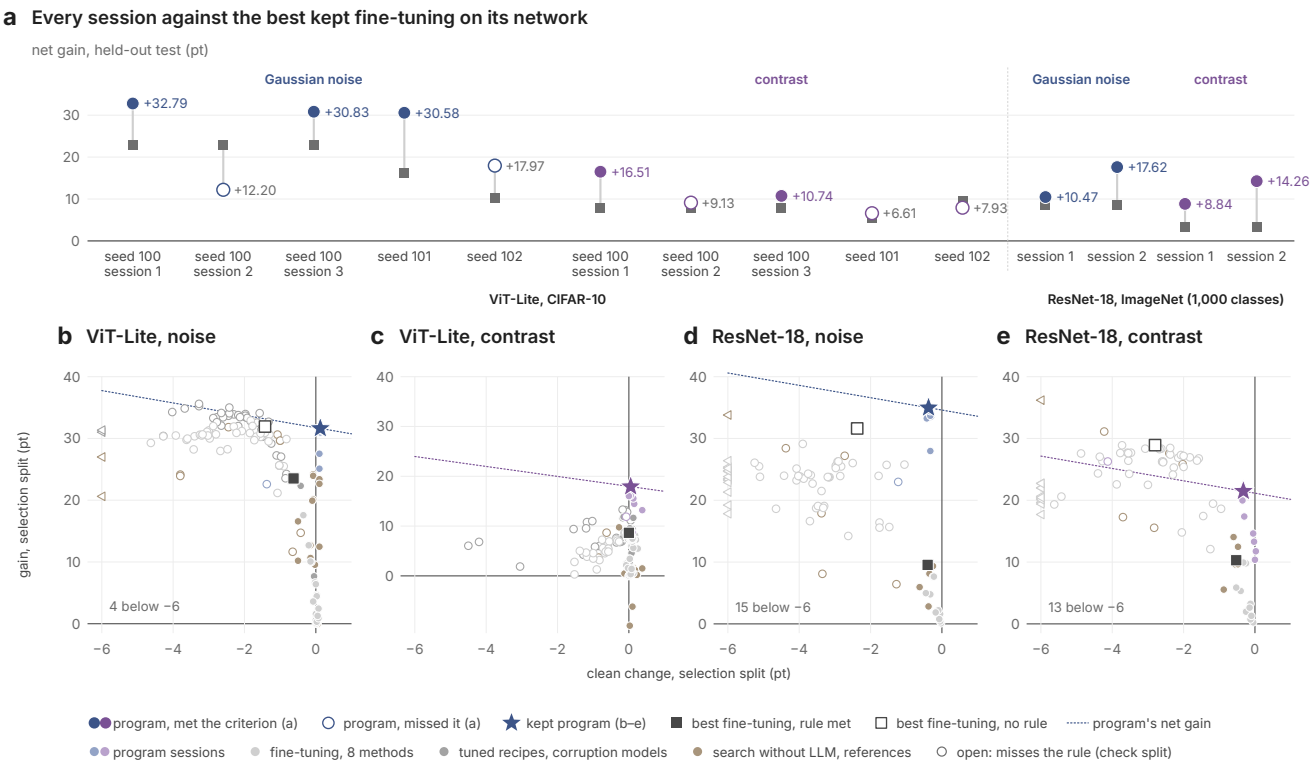


Figure 7. **Program gains and fine-tuning trade-offs across sessions and retention constraints.** (a) Net gain of every program session against the best kept fine-tuning on the same network (held-out test sets). (b–e) Gain against clean change for every candidate of every method on the four CIFAR-10 instances (selection split); star, the kept program; squares, the best fine-tuning with (filled) and without (open) the retention rule.

the seed-100 network, read on the selection split against the bar of +17.075. With labeled examples the program reaches +31.75, and with the condition labels withheld +33.37. From the capability's name alone, a program declared in a session without checks and scored once afterwards reaches +15.10, and a session with the same sentence and 8 checks +9.43. The session given only the noise formula is in Sec. A.11. Together, these sessions show that programs can be constructed from several forms of task information, including condition images without their labels and a one-sentence description of the capability. Test-time adaptation on the same unlabeled images, kept by the same rule, leaves the network unchanged (Tent [6]) or reaches +0.025 (SAR [16]). On SCAN, from the grammar, the programs reach net gains of +99.84 and +72.02 exact-match points, where fine-tuning on the same rows keeps +1.21 and +2.61 (selection split). Fine-tuning on commands generated from the grammar reaches +99.71 and +83.68 by changing 152,718 entries. Because the grammar can generate commands in the evaluation set, grammar-generated fine-tuning is a rule-to-data reference rather than a standard held-out supervised baseline. Both methods are given the same rule.

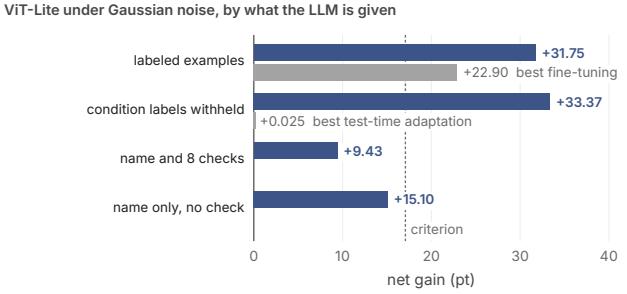


Figure 8. **A program can be written from several forms of information.** Net gain of the program on the ViT-Lite under Gaussian noise for each form of what the LLM is given (blue), and of the best other method given the same information (gray). The dashed line is the criterion fixed before these sessions. Condition labels withheld: the LLM sees predictions in place of the labels of the corrupted images, which are selected as in the labeled setting; the checks are labeled. One session per level; selection split.

The programs write the rule into the network directly, with 257 and 25,056 weights.

A.6. What programs write and what carries the gain

Table 9 characterizes the computation each reported program implements, from its code and its stated hypothesis; for four programs, interventions on the network test which parts carry the gain. In the Gaussian-noise program, removing the block-2 gate lowers the net gain from +31.75 to +10.05, removing all three gates returns the network to its original accuracy, and holding them open costs 5.03 points of clean accuracy (Fig. 3c). In the contrast program, the first-layer gain alone adds +10.95 and the gated correction alone +9.63, and together they reach +17.93 (Fig. 3d). The SCAN add-primitive program changes the embedding row of *jump* (128 entries), the output row of its action (128) and that row's bias (1), and leaves every layer between them unchanged. The embedding change alone installs nothing, the output change alone reaches 26.90% exact match, and together they reach 99.84% (selection split). With the embedding change alone, every action slot of *jump* is decoded as *walk*, *look* or *run*, and the hidden states there lie at the center of the states the original network computes for those three words, at a fifth of their spread at the input of the output layer. The new behavior thus depends on coordinated input and output edits, neither of which alone reproduces the compositional capability: the new primitive enters through the composition the network already computes, and the output row names it. In the BabyAI program, a rule written outside the network from its three parts matches the edited policy on each of 2,000 episodes (Sec. 4.2).

Table 10 gives every control of the two ViT-Lite programs discussed in Sec. 4.2, and one more alternative: an unconditional feature alignment, which applies its correction to every input, reaches +7.18 under noise, against +31.75 for the gated program.

A.7. Where the programs act

This analysis examines whether an installed computation affects only its target condition or also changes behavior under other corruptions. We read the four kept programs of the ViT-Lite and the CIFAR-10 ResNet-18, and the kept fine-tuning of the same conditions, on clean images and 25 corrupted versions of the test set: each CIFAR-10-C corruption at severity 5, and Gaussian noise and contrast at severities 1 to 4 (Fig. 9). The effect directions are stable across splits: 197 of the 208 changes carry the same label on the test sets as on the selection split, and none moves between help and harm. Each Gaussian-noise program helps on the whole noise family, at every severity and under shot, impulse and speckle noise, and on frost. The ViT-Lite program changes no other set by more than 0.27 points and harms none, where its fine-tuning harms 6. Its block-2 gate opens on 27% to 100% of patch tokens on the sets the program helps and on at most 9% elsewhere (Spearman 0.77 between gate share and change over the 25 sets, selection split). The ResNet-18 program, which writes no gate, helps on 14 sets and harms 5: contrast at severities 2 to 5, and fog. In each contrast program, the always-on first-layer gain and the gated part differ in reach. Together they help on the program's own condition, the three blurs and fog, and harm every noise set: 12 sets on the ViT-Lite and 10 on the ResNet-18, against 2 and 1 for fine-tuning. Applied alone to the original network, the gain reproduces 0.92 to 1.25 of the program's change on each of these sets, and the gated part alone changes none of them by more than 0.68 points (selection split). Clean accuracy and robustness to other corruptions are thus separate properties, and the first-layer gain governs the second. Without the gain, each program keeps 0.54 (ViT-Lite) and 0.67 (ResNet-18) of its gain on its own condition. On their own condition at severity 5, all four programs exceed their fine-tuning, by 9.2 to 25.7 points.

A.8. Rerunning programs on other networks

This section gives the per-network measurements behind Sec. 4.3 and examines the role of weight coordinates (Tab. 11). On the five ViT-Lites trained from other seeds, the Gaussian-noise program gains +27.55 to +30.54, 0.87 of its gain on its own network, and the contrast program keeps 0.92. Fine-tuning recipes retain comparable fractions of their source-network gains but reach lower absolute gains in the evaluated transfers: the best fine-tuning configuration of the seed-100 network, applied unchanged, keeps 0.89 and 0.98 of a lower gain (+20.79 and +8.09), and repeating the fine-tuning selection on each network gives +10.44 and +6.57. Three measurements locate the transfer advantage. First, the program reselects locations and parameters on each network: on one network the noise program gates blocks 2, 3 and 6 instead of 2, 4 and 6, and across the five networks its block-2 threshold ranges from -7.0 to -11.3, against -10.35 on the original. Second, the tested activation-based alignment does not recover a fixed edit: after unit-by-unit alignment the units the program changes coincide with the original's in 3 of 28 positions across the five networks, and the aligned copies of the first program of each condition recover -0.01 and +4.17 (selection split). Third, a fixed edit transfers well when networks share coordinates. Added unchanged to independently trained ViT-Lites, the edits of the ten programs gain -1.42 to +0.01 (selection split, Tab. 7). On five ResNet-18s fine-tuned from the same ImageNet weights, the copied edits carry 96% and 87% of the rerun gain, and rerunning gains more (Fig. 10, selection split).

A.9. Successive updates

Across twelve second-round sessions on the ViT-Lite, the earlier condition is preserved within the reported tolerance, and the size of the added capability differs by update order and session (Tab. 13). On the ViT-Lite, the earlier condi-

Network, condition, information	What the program writes	Weights
ViT-Lite, Gaussian noise, labeled*	frees two little-used MLP units in each of blocks 2, 4 and 6 and rewrites them as ramp gates that subtract the measured noise offset from tokens above the clean range	3,846
ViT-Lite, Gaussian noise, condition labels withheld	estimates the noise strength from the images' excess neighbor-difference energy and chains saturating gates in the least-used MLP units of blocks 2 to 4 and the last block, each triggered by a flag the previous gate writes	19,724
ViT-Lite, Gaussian noise, name only	gives the 16 block-0 MLP units most driven by noise twin units that cancel the mean the noise induces through the GELU	24,848
ViT-Lite, contrast reduction, labeled*	raises the zero-mean part of the patch-embedding filters by a factor of 1.45 and adds a gated rank-6 correction in eight freed units of block 2	16,648
ResNet-18 (CIFAR-10), Gaussian noise, labeled	estimates the noise level from the images' high-frequency energy, applies a Wiener shrinkage to each convolution in forward order and shifts the following BatchNorm means to keep clean pre-activations; no gate	1,718,057
ResNet-18 (CIFAR-10), contrast reduction, labeled	raises the zero-mean part of the stem convolution by 35% and adds gates on a whitened low-contrast direction in two late residual blocks	37,192
ResNet-18 (ImageNet), Gaussian noise, labeled	estimates the noise from pixel statistics of the given images, synthesizes noisy copies of the clean ones and removes the noisy-minus-clean activation shift at the inputs of two residual stages and of the classifier, folded into their weights and BatchNorm statistics	2,152,928
ResNet-18 (ImageNet), contrast reduction, labeled	measures the contrast factor from pixel statistics and turns the two least-used units of several residual blocks into gates that open on low-contrast inputs and add the activation shift that restores them	33,176
BabyAI policy, pick up, 8 demonstrations*	copies the embeddings of *go* and *to* into *pick* and *up*, builds from dead units an exact detector for a faced cell of the named color and type that triggers the pickup action, and adds a second detector that stops the policy from pressing forward into a blocking object	21,482
SCAN Transformer, add primitive, rule*	moves the embedding of *jump* to the centroid of those of *walk*, *look* and *run* and adds an output row and bias for its action that win where the actions of those three tie	257
SCAN Transformer, around right, rule	writes rank-one edits of attention values in blocks 2 to 4 along the network's own right-minus-left turn direction	25,056

Table 9. **What each kept program writes.** The program reported in Sec. 4 for each network, condition and level of information, described from its code and its stated hypothesis, with the number of weights it changes. *Programs whose parts were tested by intervention on the network (Sec. A.6); the other descriptions come from code and stated hypotheses.

tion moves by −0.28 to +0.23 points in every second-round session, and clean accuracy stays within 1.47 points of the original network. With Gaussian noise first, four of six sessions add at least 10 points of contrast robustness. With contrast first, all six add at least 10 points of noise robustness. Giving a session the list of entries round 1 changed leaves this rate unchanged (two of three and three of three, as without it).

A.10. Capabilities beyond the network's identified computation

The programs of Sec. 4 build each capability on computation the network already has. Two additional settings probe cases in which the target capability is not directly supported by the computations or representations identified in the original network: a new operation that must also run inside the old ones, and new classes that the network's features do not separate. In both, the program adds the capability as a separate component: a lookup table beside the old operations, and prototype rows on the base features. Each is one session, read on the selection split.

A.10.1. A new operation on a synthetic sequence task

From a symbolic rule, a weight program installs a new operation with 100% exact match on inputs of length 4 to 8 and 92.75% on inputs of length 9 to 12, while preserving the original operations. The sequence Transformer (width 64, 205,008 weights) is trained on eight symbolic operations and receives a new operation bound to a symbol permutation. The LLM is given only its symbol table, with no row of the new operation, and 64 rows of the old ones. It does so in one check, from 0%; the old operations stay at 100%, and none of their 4,000 selection rows changes. Changing 4,465 weights (2.18%), it flags the new operation along the residual stream's lowest-variance direction, routes the flag to the output positions through one attention head, and turns 16 little-used last-layer MLP units into a lookup table. The installed operation does not run inside the existing operations: composed with an old operation, the written lookup reaches 5.95% exact match, whereas fine-tuning on rows generated from the same rule reaches 100% on both lengths and on

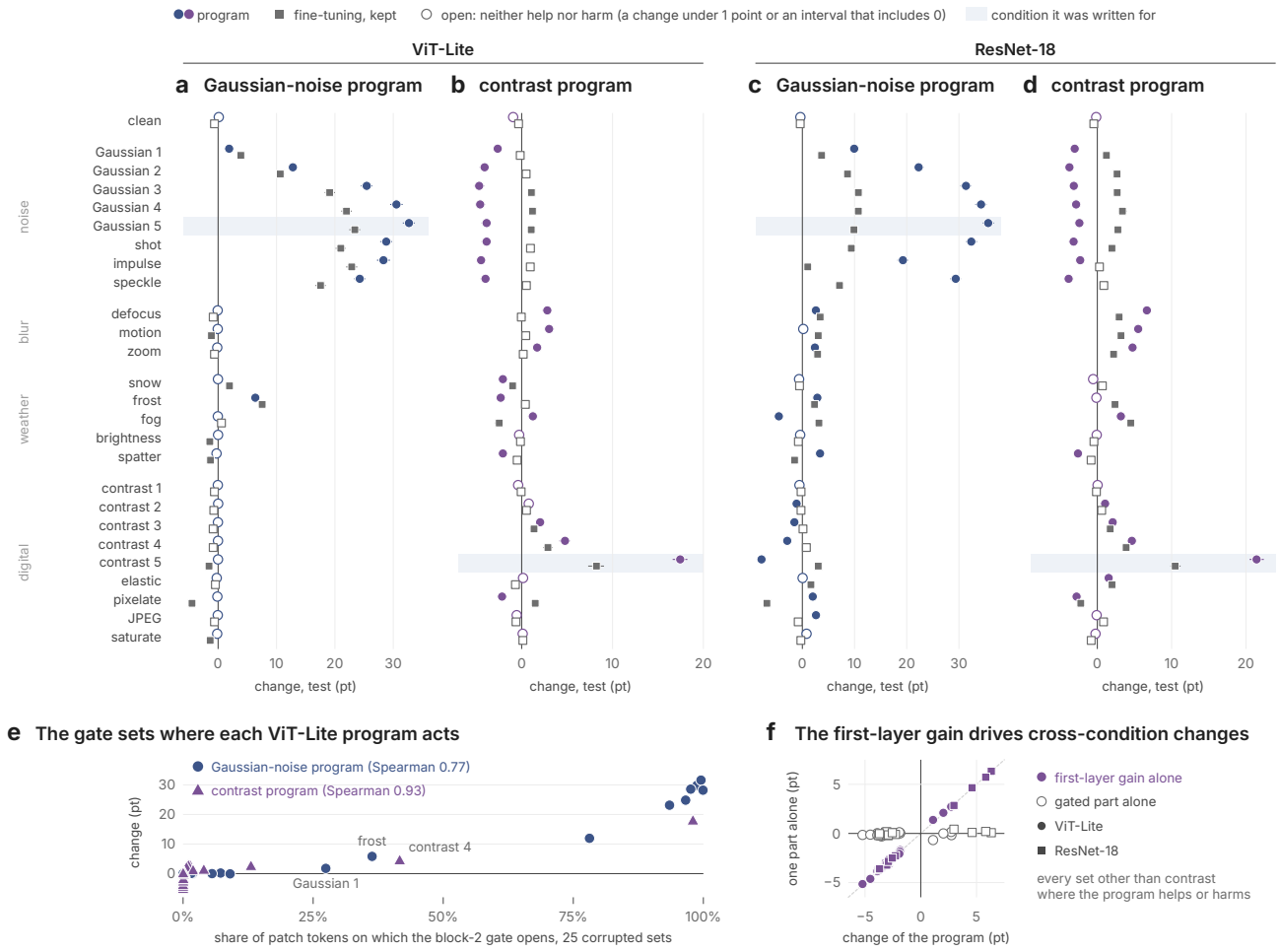


Figure 9. **Installed computations produce distinct target-condition and cross-condition effects.** (a–d) Accuracy change against the original network on clean images and on 25 corrupted versions of the CIFAR-10 test set, for the kept program and the kept fine-tuning of the same condition (Tab. 3), with paired 95% bootstrap intervals: Gaussian noise and contrast at severities 1 to 5 and the other CIFAR-10-C corruptions at severity 5, grouped by family. Filled: help or harm, a change of at least 1 point whose interval excludes 0. Each program was written for severity 5 of its condition (shaded). Held-out test sets. (e) Share of patch tokens on which the block-2 gate of each ViT-Lite program opens, against the program's change, on the 25 corrupted sets. (f) The first-layer gain determines the cross-condition changes: the two contrast programs split into their always-on first-layer gain and their gated part, each applied alone to the original network, on every set other than contrast reduction where the program helps or harms. Selection split in (e) and (f).

the composition, at -0.22 points on the old operations. On SCAN, where the program writes into the input and output rows and leaves the composition to the network, the new primitive runs inside the old compositions (Sec. A.6).

A.10.2. New classes on a small CNN

This experiment evaluates whether a weight program can add five new classes to a classifier whose features do not clearly separate them: four of the five new classes lie inside old classes in the network's features. On the few-shot class-incremental protocol of CEC [49] on CIFAR-100 (60 base classes, then five new classes per session from five labeled images each), one session wrote the five classes of session 2 into a ResNet-20 with a cosine head. The kept program writes prototype rows with margins and keeps the base classes within 0.78 points. It reaches 17.20% on the new classes and a class-balanced 64.55% on all seen classes, against 21.70% and 65.11% for imprinted prototypes, the strongest reference, and a criterion of 26.70% and 65.11%. In this single session, the generated programs stay within a prototype-based update family and do not reach the criterion fixed before the session.

	Gaussian	contrast
Weight program (WeightCode)	+31.75	+17.98
Ablations of the written program		
block-2 gate removed	+10.05	–
gates held open (clean −5.03)	+19.43	–
two random units of the block (mean of 5)	+31.86	–
Alternatives with the same examples, checks and rule		
gate from the plain mean difference	+31.13	+5.40
unconditional feature alignment	+7.18	–
gradient on its entries, from the original	+3.60	+0.72
gradient on its entries, from its values	+33.35	+18.00
the LLM without weight values	+10.78	+11.18
generic gated search, no LLM	+24.12	unchanged

Table 10. **What carries the gain.** Net gain in points on the ViT-Lite on the selection split. Interventions modify parts of the kept program to test their causal contributions and are not reselected with checks. Alternative constructions receive the program's examples and 8 checks (6 for the plain-mean gate, which under contrast reduction runs the noise program's gate form unchanged) and are kept by the same rule; "unchanged" means that no configuration met the bound. Gradient rows train exactly the 3,846 (Gaussian) or 16,648 (contrast) entries the program changes, with the fine-tuning grid. The LLM row is a session of the same LLM given the parameter names and shapes, the forward code, the examples and the checks, but no weight value or network output. The search is a post hoc family of gated corrections, written after reading the programs. The two groups answer different questions: what carries the installed gain, and whether other constructions reproduce it.

	reused	mean gain	share	clean
Gaussian	program	**+28.57**	0.87	−0.19 to +0.07
	FT recipe	+20.79	0.89	−0.85 to −0.31
	FT re-selected	+10.44	0.45	≥ -1.0
Contrast	program	**+16.04**	0.92	−1.10 to −0.47
	FT recipe	+8.09	0.98	−0.55 to −0.15
	FT re-selected	+6.57	0.80	≥ -1.0

Table 11. **Rerun on five independently trained networks, each program keeps most of its source-network gain.** Programs written on the seed-100 ViT-Lite run with no LLM call on five ViT-Lites trained from other seeds, each with its own 64 labeled examples (Gaussian-noise gains +30.54, +27.55, +27.73, +29.09, +27.92). FT recipe: the best kept fine-tuning configuration on the seed-100 network, applied unchanged; FT re-selected: the full fine-tuning selection repeated on each network. Share: mean gain on the new networks over the gain on the source network. Points on the held-out test sets.

A.11. Sessions that size edits from finite differences

Three exploratory sessions set the size of their edits from finite differences of a label-free output quantity, which the main protocol excludes: the ViT-Lite session given only the noise formula, and rounds 1 and 3 of a ResNet-20

	reused	mean gain	share	clean
Gaussian	program	**+28.14**	0.89	−0.07 to 0.00
	FT recipe	+20.71	0.88	−0.95 to −0.50
	FT re-selected	+10.33	0.44	≥ -1.0
Contrast	program	**+17.14**	0.96	−1.20 to −0.65
	FT recipe	+8.91	1.03	−0.47 to +0.18
	FT re-selected	+7.19	0.83	≥ -1.0

Table 12. **Table 11 on the selection split.** Programs written on the seed-100 ViT-Lite run with no LLM call on five ViT-Lites trained from other seeds, each with its own 64 labeled examples (Gaussian-noise gains +30.03, +27.18, +27.50, +28.55, +27.43). FT recipe: the best kept fine-tuning configuration on the seed-100 network, applied unchanged; FT re-selected: the full fine-tuning selection repeated on each network. Share: mean gain on the new networks over the gain on the source network. Points on the selection split.

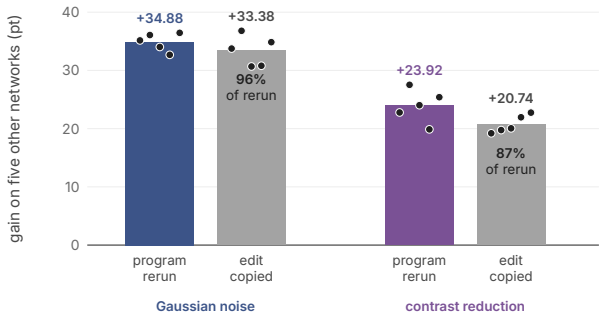


Figure 10. **On ResNet-18s fine-tuned from the same ImageNet weights, rerunning a program gains more than copying its edit.** Gain (points) of the two ResNet-18 programs rerun on, and of their edits copied to, five ResNet-18s fine-tuned from the same ImageNet weights (dots, one per network; bars, means; percentages, the copied edit's share of the rerun gain). Selection split.

stream. Round 4 of the stream starts from the values round 3 set. The automatic audit, which checks finite differences of label-scored losses, did not detect these operations; they were identified by subsequent code inspection. These exploratory results are excluded from the evidence supporting the protocol-constrained claims of Sec. 4.

The noise formula. In an exploratory session given the formula that generates Gaussian noise and 64 labeled clean images, with no noisy image, the program synthesizes noisy copies of the clean images and shrinks the low signal-to-noise components of the ViT-Lite's patch embedding and the MLP units that respond to noise. It sets each shrink fraction from a finite-difference quotient of the divergence between the network's outputs on clean and noisy copies,

Order	Edit log	added	earlier	clean	met
Gaussian first	none (first)	+9.12	−0.17	−0.25	missed
	none	+12.42	−0.02	−0.02	met
	none	+14.50	+0.01	−0.21	met
	log	+5.23	−0.09	+0.15	missed
	log	+12.80	+0.23	−1.47	met
	log	+13.42	−0.01	−0.80	met
contrast first	log	+37.65†	−0.05†	+0.15†	met
	none	+35.98†	−0.28†	+0.05†	met
	log	+35.90†	−0.20†	+0.03†	met
	none	+30.93†	−0.10†	+0.08†	met
	log	+34.00†	+0.03†	+0.00†	met
	none	+37.70†	−0.25†	+0.08†	met

Table 13. **Every second-round session on the ViT-Lite.** Round 1 is the kept program of Tab. 3 for the first condition; each row is one LLM session that writes round 2 for the second condition on the network round 1 produced. Added: gain on the second condition from round 1 to round 2; earlier: change on the first condition over the same round; clean: clean change from the original network. Edit log: the session also receives the list of entries round 1 changed. Met: all three criteria fixed before the sessions hold, added at least 10, earlier at least −3 and clean at least −2.0. Points on the held-out test sets; †: selection split.

scaled by one overall factor found against the clean damage. Changing 34,304 weights, it reaches a net gain of +18.27 at a clean change of −0.46 on the test set (+18.15 on the selection split). Fine-tuning on images generated from the same formula keeps no edit within the retention rule and reaches +32.07 at −2.83 without it (selection split).

The ResNet-20 stream. ResNet-20 receives Gaussian noise, defocus, snow and contrast in turn, with 64 labeled examples of each, one new program per round written for the network the earlier rounds produced, and a clean budget of 60 of 4,000 check images for the whole stream. Round 1's program sets each unit's shrink fraction from how much ablating the unit reduces a label-free output divergence on noisy inputs, divided by the square of its clean cost. Round 3's session adapts each BatchNorm layer alone, measures the change of label-free logit projections, and solves each layer's strength from it; the kept program carries the eleven values. The round-1 program also reads the wall clock and stops its sweep over units after 220 s. The reported stream uses the recorded round-1 execution, which examined 524 of its 688 units under machine load; a full sweep produces a different W_1. Rounds 2 to 4 and the selection among separately written programs below start from this recorded network state, so the stream's later rounds are tied to that execution.

In this exploratory stream, which includes finite-difference operations excluded from the main protocol, the mean gain over the four corruptions after four rounds is +10.61 at a clean change of −1.66, a stream score of +8.95 against +5.15 for the best of four fine-tuning streams under the same feedback, and every corruption ends at or above its gain after its own round (Tab. 14, Fig. 11). The round-2 program, written for defocus, improves all four corruptions (defocus +10.25, Gaussian noise +9.46, contrast +6.86 and snow +1.90 points over round 1). Later rounds add smaller gains once the stream has consumed its clean-accuracy budget: rounds 1 and 2 spend 25 and 35 of the 60 images, which leaves rounds 3 and 4 a floor of 0 and −1 images, and these rounds add +0.78 and +0.53 points on their own corruptions (selection split). Selecting among the four programs written separately for the original network reaches a stream score of +3.48, and the fixed references under the same feedback +4.84 (selection split).

Stream score after round	1 Gauss.	2 defocus	3 snow	4 contrast
Programs, one per round	**2.34**	**8.22**	**8.64**	**8.95**
Best fine-tuning stream	1.48	3.63	4.56	5.15

Table 14. **Exploratory four-round capability updates on ResNet-20, including operations outside the main program protocol (Sec. A.11).** ResNet-20 receives a new corruption each round with 64 labeled examples of it, and each round's program is written for the network the earlier rounds produced. Stream score: mean gain over all four corruptions plus the clean change, in points on the held-out test sets. After round 4 the programs' mean gain is +10.61 at clean −1.66, and every corruption ends at or above its gain after its own round. Fine-tuning streams re-select among 8 configurations each round under the same retention rule.

Stream score after round	1 Gauss.	2 defocus	3 snow	4 contrast
Programs, one per round	**2.31**	**8.06**	**8.90**	**8.97**
Best fine-tuning stream	1.18	3.54	4.35	4.97

Table 15. **Selection-split results for the exploratory four-round ResNet-20 stream (Tab. 14).** ResNet-20 receives a new corruption each round with 64 labeled examples of it, and each round's program is written for the network the earlier rounds produced. Stream score: mean gain over all four corruptions plus the clean change, in points on the selection split. After round 4 the programs' mean gain is +10.22 at clean −1.25, and every corruption ends at or above its gain after its own round. Fine-tuning streams re-select among 8 configurations each round under the same retention rule.

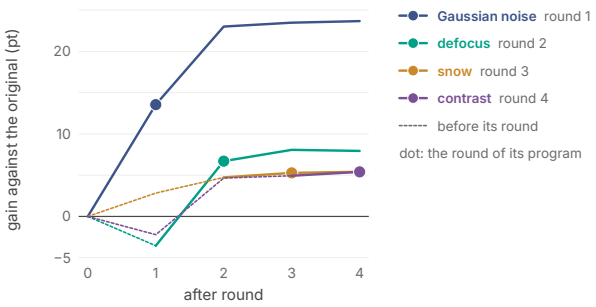


Figure 11. **Gain under each corruption in the ResNet-20 stream of Sec. A.11.** One program per round, each written for the network the earlier rounds produced. Gain against the original network after each round, dashed before the corruption's own round; dots, the round of its program. Exploratory stream that includes sessions outside the main program protocol (Sec. A.11). Held-out test sets.